

```

# ===== FILE: apps/api/dftbneu/__init__.py =====
"""dftb-neu API package."""
# ===== FILE: apps/api/dftbneu/config.py =====
"""dftb-neu 配置与数据目录。测试版可从环境变量 / 并列 cmats-lab .env 读取密钥。"""

from __future__ import annotations

import json
import os
import platform
from pathlib import Path
from typing import Any

from pydantic_settings import BaseSettings, SettingsConfigDict

def default_data_dir() -> Path:
    if platform.system() == "Windows":
        base = Path(os.environ.get("LOCALAPPDATA") or Path.home() / "AppData" / "Local")
        return base / "dftb-neu"
    return Path.home() / ".dftb-neu"

def _repo_root() -> Path:
    """仓库根，或安装包 resources 根（含 api / engines / web / templates）。"""
    env_root = os.environ.get("DFTB_NEU_ROOT", "").strip()
    if env_root:
        p = Path(env_root)
        if p.is_dir():
            return p
    here = Path(__file__).resolve()
    # 打包: resources/api/dftbneu/config.py → resources
    packaged = here.parents[2]
    if (packaged / "api").is_dir() and (packaged / "engines").is_dir():
        return packaged
    # 开发: apps/api/dftbneu/config.py → 仓库根
    return here.parents[3]

def _load_dotenv_file(path: Path) -> dict[str, str]:
    out: dict[str, str] = {}
    if not path.is_file():
        return out
    try:
        for line in path.read_text(encoding="utf-8").splitlines():
            line = line.strip()
            if not line or line.startswith("#") or "=" not in line:
                continue
            k, v = line.split("=", 1)
            k, v = k.strip(), v.strip().strip('\'').strip('"')
            if k:
                out[k] = v
    
```

```

except Exception:
    return {}
return out

def bootstrap_env_files() -> None:
    """把仓库 .env 与并列 cmats-lab/.env 中的键注入 os.environ（不覆盖已有）。

    分发策略：DeepSeek/通义 Key 不从并列 cmats-lab 自动导入（由本机设置页配置）；
    Materials Project Key 可从 cmats-lab/.env 导入，便于结构拉取。
    """
    root = _repo_root()
    files = [root / ".env"]
    sibling = root.parent / "cmats-lab" / ".env"
    if sibling.is_file():
        files.append(sibling)
    # 映射 CMATS_* → DFTB_NEU_*（仅当后者未设置）
    alias = {
        "CMATS_DEEPSEEK_API_KEY": "DFTB_NEU_DEEPSEEK_API_KEY",
        "CMATS_DEEPSEEK_BASE_URL": "DFTB_NEU_DEEPSEEK_BASE_URL",
        "CMATS_DEEPSEEK_MODEL": "DFTB_NEU_DEEPSEEK_MODEL",
        "CMATS_LLM_API_KEY": "DFTB_NEU_LLM_API_KEY",
        "CMATS_LLM_BASE_URL": "DFTB_NEU_LLM_BASE_URL",
        "CMATS_LLM_MODEL": "DFTB_NEU_LLM_MODEL",
        "CMATS_LLM_PROVIDER": "DFTB_NEU_LLM_PROVIDER",
        "CMATS_MP_API_KEY": "DFTB_NEU_MP_API_KEY",
    }
    # 并列实验室 .env 仅允许导入 MP / 非密钥类；禁止带入 LLM Key
    sibling_allow = {
        "CMATS_MP_API_KEY",
        "DFTB_NEU_MP_API_KEY",
        "CMATS_DEEPSEEK_BASE_URL",
        "CMATS_DEEPSEEK_MODEL",
        "CMATS_LLM_BASE_URL",
        "CMATS_LLM_MODEL",
        "CMATS_LLM_PROVIDER",
        "DFTB_NEU_DEEPSEEK_BASE_URL",
        "DFTB_NEU_DEEPSEEK_MODEL",
        "DFTB_NEU_LLM_BASE_URL",
        "DFTB_NEU_LLM_MODEL",
        "DFTB_NEU_LLM_PROVIDER",
        "DFTB_NEU_CLASSROOM_HUB_URL",
        "DFTB_NEU_LICENSE_REQUIRED",
        "DFTB_NEU_HOST",
        "DFTB_NEU_PORT",
    }
    sibling_deny_substrings = ("API_KEY", "SECRET", "TOKEN", "PASSWORD")
    for path in files:
        data = _load_dotenv_file(path)
        is_sibling = path.resolve() == sibling.resolve() if sibling.is_file() else
False
        for k, v in data.items():

```

```

    if not v:
        continue
    if is_sibling:
        if k not in sibling_allow and any(s in k.upper() for s in
sibling_deny_substrings):
            continue
        if "DEEPSEEK_API_KEY" in k.upper() or "LLM_API_KEY" in k.upper():
            continue
    if k not in os.environ:
        os.environ[k] = v
    mapped = alias.get(k)
    if mapped and mapped not in os.environ:
        if is_sibling and (
            "DEEPSEEK_API_KEY" in mapped.upper() or "LLM_API_KEY" in
mapped.upper()
        ):
            continue
        os.environ[mapped] = v

```

```
bootstrap_env_files()
```

```

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_prefix="DFTB_NEU_", extra="ignore")

    host: str = "127.0.0.1"
    port: int = 8765
    data_dir: str = ""
    wsl_distro: str = ""

    # DeepSeek (Key 仅来自本机设置页 / 本机环境变量; 默认不走课堂中心)
    llm_provider: str = "deepseek"
    deepseek_api_key: str = ""
    deepseek_base_url: str = "https://api.deepseek.com"
    deepseek_model: str = "deepseek-v4-flash"
    llm_api_key: str = ""
    llm_base_url: str = "https://dashscope.aliyuncs.com/compatible-mode/v1"
    llm_model: str = "qwen3.7-plus"

    # Materials Project (结构拉取; 登录后可经课堂中心代理, 或本机 MP Key)
    mp_api_key: str = ""

    # 课堂中心: 默认指向课堂服务; 可用环境变量覆盖
    classroom_hub_url: str = "https://desktop-ibhgp7g.tailcc9705.ts.net"
    license_required: bool = True
    license_offline_grace_hours: float = 12.0

    @property
    def root(self) -> Path:
        return _repo_root()

```

```

@property
def store(self) -> Path:
    p = Path(self.data_dir) if self.data_dir else default_data_dir()
    p.mkdir(parents=True, exist_ok=True)
    (p / "projects").mkdir(exist_ok=True)
    (p / "jobs").mkdir(exist_ok=True)
    return p

@property
def config_path(self) -> Path:
    return self.store / "config.json"

@property
def templates_dir(self) -> Path:
    return self.root / "templates"

settings = Settings()

def load_user_config() -> dict[str, Any]:
    path = settings.config_path
    if not path.is_file():
        return {}
    try:
        return json.loads(path.read_text(encoding="utf-8"))
    except Exception:
        return {}

def save_user_config(data: dict[str, Any]) -> None:
    path = settings.config_path
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(data, ensure_ascii=False, indent=2), encoding="utf-8")
# ===== FILE: apps/api/dftbneu/db.py =====
"""简易课题存储（JSON 文件）。"""

from __future__ import annotations

import json
import uuid
from datetime import datetime
from pathlib import Path
from typing import Any, Optional

from .config import settings

def _now() -> str:
    return datetime.now().isoformat(timespec="seconds")

```

```

def projects_dir() -> Path:
    d = settings.store / "projects"
    d.mkdir(parents=True, exist_ok=True)
    return d

def project_dir(project_id: str) -> Path:
    d = projects_dir() / project_id
    d.mkdir(parents=True, exist_ok=True)
    (d / "figures").mkdir(exist_ok=True)
    (d / "artifacts").mkdir(exist_ok=True)
    return d

def _meta_path(project_id: str) -> Path:
    return project_dir(project_id) / "project.json"

def create_project(title: str, idea: Optional[dict] = None) -> dict[str, Any]:
    pid = uuid.uuid4().hex[:12]
    proj = {
        "id": pid,
        "title": title or "DFTB+ 课题",
        "created_at": _now(),
        "phase": "draft",
        "idea": idea or {},
        "protocol": {},
        "job": None,
        "assets": [],
        "manuscript": {},
        "activity": [],
    }
    _meta_path(pid).write_text(json.dumps(proj, ensure_ascii=False, indent=2),
encoding="utf-8")
    return proj

def get_project(project_id: str) -> Optional[dict[str, Any]]:
    p = _meta_path(project_id)
    if not p.is_file():
        return None
    return json.loads(p.read_text(encoding="utf-8"))

def update_project(project_id: str, **kwargs: Any) -> Optional[dict[str, Any]]:
    proj = get_project(project_id)
    if not proj:
        return None
    proj.update(kwargs)
    proj["updated_at"] = _now()
    _meta_path(project_id).write_text(json.dumps(proj, ensure_ascii=False, indent=2),
encoding="utf-8")

```

```

return proj

def list_projects() -> list[dict[str, Any]]:
    rows = []
    for p in projects_dir().iterdir():
        if p.is_dir() and (p / "project.json").is_file():
            meta = json.loads((p / "project.json").read_text(encoding="utf-8"))
            job = meta.get("job") or {}
            protocol = meta.get("protocol") or {}
            structure = meta.get("structure") or {}
            rows.append(
                {
                    "id": meta.get("id"),
                    "title": meta.get("title"),
                    "phase": meta.get("phase"),
                    "created_at": meta.get("created_at"),
                    "job": {"job_id": job.get("job_id"), "status": job.get("status")}
                    if job else None,
                    "kind": protocol.get("kind"),
                    "sk_set": protocol.get("sk_set"),
                    "has_structure": bool(structure.get("poscar") or
                    structure.get("gen")),
                    "lesson_id": protocol.get("lesson_id"),
                }
            )
    rows.sort(key=lambda x: x.get("created_at") or "", reverse=True)
    return rows

def delete_project(project_id: str) -> bool:
    import shutil

    root = projects_dir() / project_id
    if not root.is_dir():
        return False
    shutil.rmtree(root, ignore_errors=True)
    return not root.exists()

def append_activity(project_id: str, message: str, phase: str = "") -> None:
    proj = get_project(project_id)
    if not proj:
        return
    act = list(proj.get("activity") or [])
    act.append({"at": _now(), "message": message, "phase": phase or proj.get("phase")})
    update_project(project_id, activity=act[-100:], phase=phase or proj.get("phase"))
# ===== FILE: apps/api/dftbneu/main.py =====
"""dftb-neu FastAPI 应用。"""

from __future__ import annotations

```

```

import sys
from pathlib import Path

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import FileResponse
from fastapi.staticfiles import StaticFiles

def _detect_root() -> Path:
    here = Path(__file__).resolve()
    packaged = here.parents[2]
    if (packaged / "api").is_dir() and (packaged / "engines").is_dir():
        return packaged
    return here.parents[3]

# engines on path
_ROOT = _detect_root()
_ENG = _ROOT / "engines" / "dftb_agent"
if str(_ENG) not in sys.path:
    sys.path.insert(0, str(_ENG))
if str(Path(__file__).resolve().parent.parent) not in sys.path:
    sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

from dftbneu.routers import (
    assets,
    chat,
    courses,
    deploy,
    jobs,
    license as license_router,
    projects,
    settings as settings_router,
    status,
    structure,
)

app = FastAPI(title="DFTB Neu", version="0.2.1")
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(settings_router.router, prefix="/api")
app.include_router(deploy.router, prefix="/api")
app.include_router(chat.router, prefix="/api")
app.include_router(courses.router, prefix="/api")
app.include_router(projects.router, prefix="/api")
app.include_router(jobs.router, prefix="/api")

```

```

app.include_router(structure.router, prefix="/api")
app.include_router(assets.router, prefix="/api")
app.include_router(status.router, prefix="/api")
app.include_router(license_router.router, prefix="/api")

@app.middleware("http")
async def classroom_license_gate(request, call_next):
    path = request.url.path or ""
    if not path.startswith("/api/"):
        return await call_next(request)
    open_prefixes = (
        "/api/health",
        "/api/license",
        "/api/settings",
        "/api/status",
    )
    if any(path == p or path.startswith(p + "/") for p in open_prefixes):
        return await call_next(request)
    import time

    from fastapi.responses import JSONResponse

    from dftbneu.services import license as lic

    if not lic.license_required():
        return await call_next(request)
    st = lic.public_license_status()
    if not st.get("activated"):
        return JSONResponse(
            {"detail": "尚未登录或会话已失效", "license": st},
            status_code=401,
        )
    # 本地过期立即拒绝; 超过 15 秒未在线校验则向中心重验 (教师清密码后尽快失效)
    cfg_last = float(st.get("last_ok_at") or 0)
    if not cfg_last or (time.time() - cfg_last) > 15:
        check = await lic.check_session(allow_offline_grace=True)
        if not check.get("ok"):
            return JSONResponse(
                {
                    "detail": check.get("message") or "课堂许可无效, 请重新登录",
                    "license": lic.public_license_status(),
                },
                status_code=401,
            )
    return await call_next(request)

@app.get("/api/health")
def health():
    return {"ok": True, "service": "dftb-neu"}

```

```

# 打包: resources/web; 开发: apps/web[/dist]
_web_candidates = [
    _ROOT / "web",
    _ROOT / "apps" / "web" / "dist",
    _ROOT / "apps" / "web",
]

_web_dir = next((p for p in _web_candidates if (p / "index.html").is_file()), None)
if _web_dir is not None:
    _NO_CACHE = {
        "Cache-Control": "no-store, no-cache, must-revalidate, max-age=0",
        "Pragma": "no-cache",
        "Expires": "0",
    }

    if (_web_dir / "app.js").is_file() and not (_web_dir / "assets").is_dir():
        # 源码静态页 (无 vite dist)
        @app.get("/")
        def index():
            return FileResponse(_web_dir / "index.html", headers=_NO_CACHE)

        @app.get("/static/{file_path:path}")
        def static_file(file_path: str):
            target = (_web_dir / file_path).resolve()
            if not str(target).startswith(str(_web_dir.resolve())) or not
target.is_file():
                from fastapi import HTTPException

                raise HTTPException(404)
            return FileResponse(target, headers=_NO_CACHE)

    else:
        app.mount("/", StaticFiles(directory=str(_web_dir), html=True), name="web")

def main():
    import uvicorn
    from dftbneu.config import settings

    uvicorn.run("dftbneu.main:app", host=settings.host, port=settings.port,
reload=False)

if __name__ == "__main__":
    main()

# ===== FILE: apps/api/dftbneu/routers/assets.py =====
from __future__ import annotations

from pathlib import Path

from fastapi import APIRouter, HTTPException, Query
from fastapi.responses import FileResponse

```

```

from dftb_engine import kind_profile as kp
from dftb_engine.service import DftbEngine

from .. import db
from ..config import load_user_config, settings

router = APIRouter(tags=["assets"])

_FIG_SKIP = {"dftb_total_energy.png", "dftb_performance.png"}

_INPUT_NAMES = {
    "dftb_in.hsd",
    "geo.gen",
    "POSCAR",
    "poscar",
    "band_meta.json",
    "run.sh",
}
# 仅保留 DFTB+ 产生的有意义输出（不含调度元数据 / runner 包装日志）
_OUTPUT_NAMES = {
    "detailed.out",
    "band.out",
    "geo_end.gen",
    "EXC.DAT",
    "dftb.log",
}
# 仅能带/DOS 类任务展示的输出
_BAND_ONLY_OUTPUTS = {"band.out", "band.dat", "band_meta.json"}

def _kind_for(name: str) -> str:
    if name in _INPUT_NAMES:
        return "input"
    if name in _OUTPUT_NAMES:
        return "output"
    low = name.lower()
    if low.endswith((".hsd", ".gen", ".poscar", ".vasp")) or low in ("poscar",):
        return "input"
    return "other"

@router.get("/projects/{project_id}/figures")
def list_figures(project_id: str):
    p = db.get_project(project_id)
    if not p:
        raise HTTPException(404, "课题不存在")
    fig_dir = db.project_dir(project_id) / "figures"
    files = []
    if fig_dir.is_dir():
        for f in sorted(fig_dir.iterdir()):
            if f.suffix.lower() == ".png" and f.name.lower() not in _FIG_SKIP:

```

```

        files.append(
            {
                "name": f.name,
                "url": f"/api/projects/{project_id}/figures/{f.name}",
                "download_url":
f"/api/projects/{project_id}/figures/{f.name}?download=1",
            }
        )
    return {"figures": files, "protocol_figures": (p.get("protocol") or
{}).get("figures") or []}

@router.get("/projects/{project_id}/figures/{name}")
def get_figure(project_id: str, name: str, download: int = Query(0)):
    if "/" in name or ".." in name:
        raise HTTPException(400, "非法文件名")
    path = db.project_dir(project_id) / "figures" / name
    if not path.is_file():
        raise HTTPException(404, "图不存在")
    if download:
        return FileResponse(path, filename=name, media_type="image/png")
    return FileResponse(path)

@router.get("/projects/{project_id}/artifacts")
def list_artifacts(project_id: str):
    if not db.get_project(project_id):
        raise HTTPException(404, "课题不存在")
    art = db.project_dir(project_id) / "artifacts"
    names = []
    if art.is_dir():
        names = sorted(f.name for f in art.iterdir() if f.is_file())
    return {"artifacts": names}

@router.get("/projects/{project_id}/artifacts/{name}")
def get_artifact(project_id: str, name: str):
    if "/" in name or ".." in name:
        raise HTTPException(400, "非法文件名")
    path = db.project_dir(project_id) / "artifacts" / name
    if not path.is_file():
        raise HTTPException(404, "资产不存在")
    return FileResponse(path)

def _project_kind(project_id: str = "", proj: dict | None = None) -> str:
    p = proj if proj is not None else (db.get_project(project_id) or {})
    return str((p.get("protocol") or {}).get("kind") or "")

def _io_name_allowed(name: str, kind: str = "") -> bool:
    """按任务类型过滤 IO 列表：优化等不展示 band.out。"""

```

```

    if name in _BAND_ONLY_OUTPUTS and not kp.shows_kpath(kind) and kind not in
kp.BAND_KINDS:
        # band_meta 作输入时也仅能带任务保留
        return False
    return True

def _ensure_local_artifacts(project_id: str) -> Path:
    """若本机 artifacts 不足, 从 WSL 作业目录补拉文本文件。"""
    art = db.project_dir(project_id) / "artifacts"
    art.mkdir(parents=True, exist_ok=True)
    proj = db.get_project(project_id) or {}
    job_id = (proj.get("job") or {}).get("job_id") or (proj.get("protocol") or
{}).get("job_id") or ""
    existing = {p.name for p in art.iterdir() if p.is_file()} if art.is_dir() else set()
    # 补写方案中的结构 / HSD (未落盘时)
    protocol = proj.get("protocol") or {}
    structure = proj.get("structure") or {}
    kind = _project_kind(proj=proj)
    if "dftb_in.hsd" not in existing:
        hsd = (protocol.get("hsd_preview") or protocol.get("hsd_default") or "").strip()
        if hsd:
            (art / "dftb_in.hsd").write_text(hsd, encoding="utf-8")
            existing.add("dftb_in.hsd")
    if "POSCAR" not in existing and structure.get("poscar"):
        (art / "POSCAR").write_text(str(structure["poscar"]), encoding="utf-8")
        existing.add("POSCAR")
    if "geo.gen" not in existing and structure.get("gen"):
        (art / "geo.gen").write_text(str(structure["gen"]), encoding="utf-8")
        existing.add("geo.gen")
    need = [
        "dftb_in.hsd",
        "geo.gen",
        "POSCAR",
        "detailed.out",
        "dftb.log",
        "runner.log",
        "geo_end.gen",
        "EXC.DAT",
        "status",
        "exit_code",
    ]
    if kind in kp.BAND_KINDS or kp.shows_kpath(kind):
        need.extend(["band_meta.json", "band.out"])
    if kind in kp.MD_KINDS or kp.shows_md_metrics(kind):
        need.append("md.out")
    if kind in kp.VIB_KINDS or kp.shows_vibrations(kind):
        need.extend(["modes.log", "vibrations.tag", "hessian.out"])
    missing = [n for n in need if n not in existing]
    if job_id and missing:
        try:
            cfg = load_user_config()

```

```

        eng = DftbEngine(wsl_distro=str(cfg.get("wsl_distro") or
settings.wsl_distro or ""))
        fetched = eng.fetch_artifacts(job_id, missing)
        for name, content in fetched.items():
            if content is None:
                continue
            (art / name).write_text(content, encoding="utf-8", errors="replace")
    except Exception:
        pass
    return art

@router.get("/projects/{project_id}/io-files")
def list_io_files(project_id: str, sync: int = Query(1)):
    """列出本次计算的输入 / 输出文本文件，供逐个查看。"""
    proj = db.get_project(project_id)
    if not proj:
        raise HTTPException(404, "课题不存在")
    job_kind = _project_kind(proj=proj)
    art = _ensure_local_artifacts(project_id) if sync else db.project_dir(project_id) /
"artifacts"
    files = []
    if art.is_dir():
        for f in sorted(art.iterdir(), key=lambda p: (_kind_for(p.name),
p.name.lower())):
            if not f.is_file():
                continue
            # 跳过体积很大的二进制
            if f.suffix.lower() in {".bin", ".tag", ".npz"}:
                continue
            if not _io_name_allowed(f.name, job_kind):
                continue
            try:
                size = f.stat().st_size
            except OSError:
                size = 0
            if size > 2_000_000:
                continue
            kind = _kind_for(f.name)
            if kind == "other":
                continue
            if f.name not in _INPUT_NAMES and f.name not in _OUTPUT_NAMES:
                # 额外结构文件可作输入；其它杂项不列出
                if kind != "input":
                    continue
            files.append(
                {
                    "name": f.name,
                    "kind": kind,
                    "kind_zh": {"input": "输入文件", "output": "DFTB+ 输出", "other": "其
它"}.get(
                        kind, "其它"

```

```

        ),
        "size": size,
        "url": f"/api/projects/{project_id}/io-files/{f.name}",
        "download_url": f"/api/projects/{project_id}/io-
files/{f.name}?download=1",
    }
)
# 输入优先、再输出
order = {"input": 0, "output": 1, "other": 2}
files.sort(key=lambda x: (order.get(x["kind"], 9), x["name"].lower()))
return {"files": files}

@router.get("/projects/{project_id}/io-files/{name}")
def get_io_file(project_id: str, name: str, download: int = Query(0)):
    if "/" in name or "." in name:
        raise HTTPException(400, "非法文件名")
    if not db.get_project(project_id):
        raise HTTPException(404, "课题不存在")
    art = _ensure_local_artifacts(project_id)
    path = art / name
    if not path.is_file():
        raise HTTPException(404, "文件不存在")
    if download:
        return FileResponse(path, filename=name, media_type="text/plain; charset=utf-8")
    try:
        text = path.read_text(encoding="utf-8", errors="replace")
    except Exception as e:
        raise HTTPException(500, f"读取失败: {e}") from e
    # 预览截断, 避免超大日志拖垮前端
    truncated = len(text) > 400_000
    if truncated:
        text = text[:400_000] + "\n\n... (内容过长, 已截断; 可点「下载」查看全文)"
    return {
        "name": name,
        "kind": _kind_for(name),
        "kind_zh": {"input": "输入文件", "output": "DFTB+ 输出", "other": "其它"}.get(
            _kind_for(name), "其它"
        ),
        "text": text,
        "truncated": truncated,
        "download_url": f"/api/projects/{project_id}/io-files/{name}?download=1",
    }

@router.get("/jobs/{job_id}/log")
def job_log(job_id: str, tail: int = 80):
    cfg = load_user_config()
    eng = DftbEngine(wsl_distro=str(cfg.get("wsl_distro") or settings.wsl_distro or ""))
    arts = eng.fetch_artifacts(job_id, ["dftb.log", "detailed.out", "status"])
    log = arts.get("dftb.log") or ""
    lines = log.splitlines()

```

```

raw_status = (arts.get("status") or "").strip()
st = raw_status.lower()
zh = {
    "pending": "排队中",
    "running": "计算中",
    "done": "已完成",
    "error": "失败",
    "cancelled": "已取消",
}.get(st, raw_status or "未知")
return {
    "job_id": job_id,
    "status": raw_status,
    "status_zh": zh,
    "log_tail": "\n".join(lines[-max(tail, 10) :]),
    "has_detailed": "detailed.out" in arts,
}
# ===== FILE: apps/api/dftbneu/routers/chat.py =====
from __future__ import annotations

from typing import Any

from fastapi import APIRouter
from pydantic import BaseModel, field_validator

from ..services import pipeline

router = APIRouter(tags=["chat"])

def _as_str(v: Any) -> str:
    if v is None:
        return ""
    return v if isinstance(v, str) else str(v)

class ChatIn(BaseModel):
    message: str = ""
    project_id: str = ""
    poscar: str = ""
    gen: str = ""
    hsd: str = ""
    confirm_submit: bool = False
    family: str = ""
    kind: str = ""

    @field_validator("message", "project_id", "poscar", "gen", "hsd", "family", "kind",
mode="before")
    @classmethod
    def _coerce_str(cls, v: Any) -> str:
        return _as_str(v)

```

```

@router.post("/chat")
async def chat(body: ChatIn):
    return await pipeline.handle_chat(
        body.message,
        project_id=body.project_id or None,
        poscar=body.poscar,
        gen=body.gen,
        hsd=body.hsd,
        confirm_submit=body.confirm_submit,
        family_hint=body.family or "",
        kind_hint=body.kind or "",
    )

@router.get("/examples")
def examples():
    return {"examples": pipeline.teaching_examples()}

@router.get("/commands")
def commands():
    return {"commands": pipeline.commands()}

@router.post("/preview")
def preview(body: ChatIn):
    return pipeline.preview_hsd(
        body.message,
        poscar=body.poscar,
        gen=body.gen,
        family_hint=body.family or "",
        kind_hint=body.kind or "",
    )

# ===== FILE: apps/api/dftbneu/routers/courses.py =====
from __future__ import annotations

from fastapi import APIRouter, HTTPException

from ..services import courses as courses_svc

router = APIRouter(tags=["courses"])

@router.get("/courses")
def list_courses():
    return {"courses": courses_svc.list_courses()}

@router.get("/courses/lessons/{lesson_id}")
def get_lesson(lesson_id: str):
    lesson = courses_svc.get_lesson(lesson_id)
    if not lesson:

```

```

        raise HTTPException(404, f"未找到课次: {lesson_id}")
    return {"lesson": lesson}

@router.post("/courses/lessons/{lesson_id}/start")
def start_lesson(lesson_id: str):
    data = courses_svc.start_lesson(lesson_id)
    if not data.get("ok"):
        raise HTTPException(404, data.get("message") or "课次不存在")
    return data

@router.post("/courses/lessons/{lesson_id}/recipe")
def run_recipe(lesson_id: str):
    """一键复现：载入结构并生成 HSD 方案。"""
    data = courses_svc.run_lesson_recipe(lesson_id)
    if not data.get("ok"):
        raise HTTPException(404, data.get("message") or "课次不存在")
    return data

# ===== FILE: apps/api/dftbneu/routers/deploy.py =====
from __future__ import annotations

from typing import Optional

from fastapi import APIRouter
from pydantic import BaseModel

from ..services import deploy as deploy_svc

router = APIRouter(tags=["deploy"])

class DeployIn(BaseModel):
    distro: str = ""
    # 前端若已通过 Electron 提权启动过 WSL, 可设为 False, 避免重复弹出 UAC
    ensure_wsl: bool = True

@router.get("/deploy/status")
def status():
    return deploy_svc.deploy_status()

@router.get("/deploy/wsl")
def wsl():
    return deploy_svc.detect_wsl()

@router.post("/deploy/ensure-wsl")
def ensure_wsl():
    return deploy_svc.ensure_wsl()

```

```

@router.post("/deploy/run")
def run(body: Optional[DeployIn] = None):
    body = body or DeployIn()
    return deploy_svc.run_deploy(distro=body.distro, ensure_wsl_first=body.ensure_wsl)
# ===== FILE: apps/api/dftbneu/routers/jobs.py =====
from __future__ import annotations

from fastapi import APIRouter

from dftb_engine.service import DftbEngine

from ..config import load_user_config, settings

router = APIRouter(tags=["jobs"])

@router.get("/jobs/{job_id}")
def job_status(job_id: str):
    cfg = load_user_config()
    eng = DftbEngine(wsl_distro=str(cfg.get("wsl_distro") or settings.wsl_distro or ""))
    return eng.get_status(job_id)

@router.post("/jobs/{job_id}/cancel")
def job_cancel(job_id: str):
    cfg = load_user_config()
    eng = DftbEngine(wsl_distro=str(cfg.get("wsl_distro") or settings.wsl_distro or ""))
    return eng.cancel_job(job_id)
# ===== FILE: apps/api/dftbneu/routers/license.py =====
from __future__ import annotations

from typing import Optional

from fastapi import APIRouter, HTTPException
from pydantic import BaseModel

from ..services import license as lic

router = APIRouter(tags=["license"])

class ActivateIn(BaseModel):
    code: str
    student_label: str = ""
    student_id: str = ""
    hub_url: str = ""

class HubSettingsIn(BaseModel):
    hub_url: str = ""
    license_required: Optional[bool] = None

```

```

class TeacherGateIn(BaseModel):
    teacher_password: str
    student_password: str
    hub_url: str = ""
    session_days: int = 30

@router.get("/license")
async def get_license():
    status = await lic.check_session(allow_offline_grace=True)
    return {"license": lic.public_license_status(), "check": status}

@router.post("/license/activate")
async def activate(body: ActivateIn):
    try:
        data = await lic.activate(
            code=body.code,
            student_label=body.student_label,
            student_id=body.student_id,
            hub=body.hub_url,
        )
        return {"ok": True, "activation": data, "license": lic.public_license_status()}
    except lic.LicenseError as e:
        raise HTTPException(403, str(e))

@router.post("/license/check")
async def check():
    return await lic.check_session(allow_offline_grace=True)

@router.post("/license/clear")
def clear():
    lic.clear_session()
    return {"ok": True, "license": lic.public_license_status()}

@router.post("/license/hub")
def set_hub(body: HubSettingsIn):
    lic.save_hub_settings(hub_url_value=body.hub_url,
        license_required_value=body.license_required)
    return {"ok": True, "license": lic.public_license_status()}

@router.post("/license/teacher/gate")
async def teacher_set_gate(body: TeacherGateIn):
    try:
        data = await lic.teacher_set_gate(
            teacher_password=body.teacher_password,

```

```

        student_password=body.student_password,
        hub=body.hub_url,
        session_days=body.session_days,
    )
    return {"ok": True, "gate": data}
except lic.LicenseError as e:
    raise HTTPException(403, str(e))

@router.get("/license/teacher/gate")
async def teacher_get_gate(teacher_password: str = "", hub_url: str = ""):
    try:
        data = await lic.teacher_get_gate(teacher_password=teacher_password,
        hub=hub_url)
        return {"ok": True, "gate": data}
    except lic.LicenseError as e:
        raise HTTPException(403, str(e))
# ===== FILE: apps/api/dftbneu/routers/projects.py =====
from __future__ import annotations

from typing import Optional

from fastapi import APIRouter, HTTPException
from fastapi.responses import FileResponse
from pydantic import BaseModel

from .. import db
from ..services import manuscript as ms_svc
from ..services import pipeline

router = APIRouter(tags=["projects"])

class CreateIn(BaseModel):
    title: str = "DFTB+ 课题"
    prompt: str = ""

class SubmitIn(BaseModel):
    poscar: str = ""
    gen: str = ""

class PollIn(BaseModel):
    force_replot: bool = False

@router.get("/projects")
def list_projects():
    return {"projects": db.list_projects()}

```

```

@router.post("/projects")
def create(body: CreateIn):
    return db.create_project(body.title, idea={"prompt": body.prompt})

@router.get("/projects/{project_id}")
def get_one(project_id: str):
    p = db.get_project(project_id)
    if not p:
        raise HTTPException(404, "课题不存在")
    return p

@router.post("/projects/{project_id}/submit")
def submit(project_id: str, body: Optional[SubmitIn] = None):
    body = body or SubmitIn()
    return pipeline.submit_project(project_id, poscar=body.poscar, gen=body.gen)

@router.post("/projects/{project_id}/poll")
def poll(project_id: str, body: Optional[PollIn] = None):
    body = body or PollIn()
    return pipeline.poll_and_finalize(project_id, force_replot=bool(body.force_replot))

@router.post("/projects/{project_id}/cancel")
def cancel(project_id: str):
    data = pipeline.cancel_project(project_id)
    if not data.get("ok"):
        raise HTTPException(400, data.get("message") or "取消失败")
    return data

@router.delete("/projects/{project_id}")
def delete(project_id: str):
    from .. import db as db_mod

    if not db_mod.get_project(project_id):
        raise HTTPException(404, "任务不存在")
    # 尽量先取消进行中的作业
    try:
        pipeline.cancel_project(project_id)
    except Exception:
        pass
    if not db_mod.delete_project(project_id):
        raise HTTPException(500, "删除失败")
    return {"ok": True, "message": "已删除方案"}

@router.post("/projects/{project_id}/manuscript")
def manuscript(project_id: str):
    try:

```

```

        return ms_svc.build_manuscript(project_id, use_llm=True)
    except ValueError as e:
        raise HTTPException(404, str(e))

@router.get("/projects/{project_id}/export/{fmt}")
def export(project_id: str, fmt: str):
    p = db.project_dir(project_id)
    if fmt == "md":
        path = p / "artifacts" / "manuscript.md"
    elif fmt == "html":
        path = p / "artifacts" / "manuscript.html"
    else:
        raise HTTPException(400, "仅支持 md / html")
    if not path.is_file():
        ms_svc.build_manuscript(project_id, use_llm=False)
    if not path.is_file():
        raise HTTPException(404, "文稿尚未生成")
    return FileResponse(path)
# ===== FILE: apps/api/dftbneu/routers/settings.py =====
from __future__ import annotations

from fastapi import APIRouter
from pydantic import BaseModel

from ..services import llm
from ..services.secrets import public_settings, save_llm_settings

router = APIRouter(tags=["settings"])

class SettingsIn(BaseModel):
    api_key: str = ""
    base_url: str = ""
    model: str = ""
    llm_provider: str = ""
    llm_api_key: str = ""
    llm_base_url: str = ""
    llm_model: str = ""
    mp_api_key: str = ""
    wsl_distro: str = ""

@router.get("/settings")
def get_settings():
    return public_settings()

@router.post("/settings")
async def post_settings(body: SettingsIn):
    out = save_llm_settings(
        api_key=body.api_key,

```

```

        base_url=body.base_url,
        model=body.model,
        llm_provider=body.llm_provider,
        llm_api_key=body.llm_api_key,
        llm_base_url=body.llm_base_url,
        llm_model=body.llm_model,
        mp_api_key=body.mp_api_key,
        wsl_distro=body.wsl_distro,
    )
    probe = await llm.probe("all")
    return {"settings": out, "probe": probe}

@router.post("/settings/probe")
async def probe_settings():
    return await llm.probe("all")
# ===== FILE: apps/api/dftbneu/routers/status.py =====
from __future__ import annotations

from fastapi import APIRouter

from ..services import deploy as deploy_svc
from ..services import llm
from ..services import license as lic
from ..services import structure_mp
from ..services.secrets import public_settings

router = APIRouter(tags=["status"])

@router.get("/status")
async def status():
    settings_pub = public_settings()
    deploy = deploy_svc.deploy_status()
    # 状态栏用轻量就绪检查, 避免每次真实调用 LLM 导致登录后卡住
    llm_probe = llm.readiness()
    license_check = await lic.check_session(allow_offline_grace=True)
    return {
        "service": "dftb-neu",
        "version": "0.1.0",
        "settings": settings_pub,
        "license": lic.public_license_status(),
        "license_check": license_check,
        "llm": {
            "ok": llm_probe.get("ok"),
            "active_provider": llm_probe.get("active_provider"),
            "results": llm_probe.get("results"),
        },
        "materials_project": structure_mp.mp_status(),
        "deploy": {
            "wsl": deploy.get("wsl"),
            "dftb": deploy.get("dftb"),

```

```

        "smoke": deploy.get("smoke"),
        "readiness": deploy.get("readiness"),
    },
}

# ===== FILE: apps/api/dftbneu/routers/structure.py =====
from __future__ import annotations

from fastapi import APIRouter, HTTPException
from pydantic import BaseModel

from ..services import structure_mp

router = APIRouter(tags=["structure"])

class MpIn(BaseModel):
    query: str = ""
    material_id: str = ""
    limit: int = 8

@router.get("/structure/status")
def status():
    return structure_mp.mp_status()

@router.post("/structure/search")
def search(body: MpIn):
    try:
        return {"results": structure_mp.search_materials(body.query, limit=body.limit
or 8)}
    except structure_mp.StructureError as e:
        raise HTTPException(400, str(e))

@router.post("/structure/fetch")
def fetch(body: MpIn):
    try:
        return structure_mp.fetch_poscar(body.query, material_id=body.material_id)
    except structure_mp.StructureError as e:
        raise HTTPException(400, str(e))

# ===== FILE: apps/api/dftbneu/services/courses.py =====
"""固体物理课次: 读取 templates/courses 并预填教学演示上下文."""

from __future__ import annotations

import json
from pathlib import Path
from typing import Any, Optional

from ..config import settings

```

```

def _courses_dir() -> Path:
    return settings.root / "templates" / "courses"

def _load_course_file(name: str = "solid_state_physics.json") -> dict[str, Any]:
    path = _courses_dir() / name
    if not path.is_file():
        return {"id": "solid_state_physics", "title": "固体物理课堂演示", "lessons": []}
    return json.loads(path.read_text(encoding="utf-8"))

def list_courses() -> list[dict[str, Any]]:
    course = _load_course_file()
    lessons = course.get("lessons") or []
    return [
        {
            "id": course.get("id"),
            "title": course.get("title"),
            "description": course.get("description"),
            "curriculum_note": course.get("curriculum_note") or "",
            "lesson_count": len(lessons),
            "lessons": [
                {
                    "id": L.get("id"),
                    "index": L.get("index"),
                    "title": L.get("title"),
                    "chapter": L.get("chapter") or "",
                    "topic": L.get("topic"),
                    "knowledge": L.get("knowledge") or "",
                    "objectives": L.get("objectives") or [],
                    "expected_observables": L.get("expected_observables") or [],
                    "quiz": L.get("quiz") or "",
                    "structure_hint": L.get("structure_hint") or "",
                    "needs_structure": bool(L.get("needs_structure")),
                    # 内部字段仍返回, 供计算管线; 前端不展示
                    "capability_family": L.get("capability_family"),
                    "kind": L.get("kind"),
                }
                for L in lessons
            ],
        }
    ]

def get_lesson(lesson_id: str) -> Optional[dict[str, Any]]:
    course = _load_course_file()
    for L in course.get("lessons") or []:
        if L.get("id") == lesson_id:
            out = dict(L)
            out["course_id"] = course.get("id")
            out["course_title"] = course.get("title")

```

```

        return out
    return None

def _read_structure(filename: str) -> str:
    if not filename:
        return ""
    path = _courses_dir() / "structures" / filename
    if not path.is_file():
        return ""
    return path.read_text(encoding="utf-8")

def start_lesson(lesson_id: str) -> dict[str, Any]:
    """返回课次教学上下文：知识点、prompt、内置结构。"""
    lesson = get_lesson(lesson_id)
    if not lesson:
        return {"ok": False, "message": f"未找到课次: {lesson_id}"}
    poscar = _read_structure(str(lesson.get("structure_file") or ""))
    chapter = lesson.get("chapter") or ""
    title = lesson.get("title") or ""
    msg = f"已载入教学演示「{title}」"
    if chapter:
        msg += f"({chapter})"
    msg += "。"
    if poscar:
        msg += " 已填入指令与结构；若要生成 HSD，请点「预览输入文件」，或回课程用「生成输入文件」。"
    elif lesson.get("needs_structure"):
        msg += " 请先准备结构。"
    return {
        "ok": True,
        "lesson": lesson,
        "prompt": lesson.get("prompt") or "",
        "poscar": poscar,
        "gen": "",
        "family": lesson.get("capability_family"),
        "kind": lesson.get("kind"),
        "needs_structure": bool(lesson.get("needs_structure")),
        "structure_ready": bool(poscar) or not lesson.get("needs_structure"),
        "expected_observables": lesson.get("expected_observables") or [],
        "message": msg,
    }

def run_lesson_recipe(lesson_id: str) -> dict[str, Any]:
    """一键复现：载入课次 + 生成 HSD 方案（不自动投递）。"""
    from . import pipeline

    base = start_lesson(lesson_id)
    if not base.get("ok"):
        return base
    if base.get("needs_structure") and not base.get("structure_ready"):

```

```

base["recipe_ready"] = False
base["message"] = (base.get("message") or "") + " 缺少结构，无法一键生成输入。"
return base

prompt = base.get("prompt") or ""
poscar = base.get("poscar") or ""
preview = pipeline.preview_hsd(
    prompt,
    poscar=poscar,
    gen="",
    family_hint=str(base.get("family") or ""),
    kind_hint=str(base.get("kind") or ""),
)
if preview.get("needs_structure") and not preview.get("ok"):
    base["preview"] = preview
    base["recipe_ready"] = False
    base["message"] = preview.get("message") or "需要结构文件。"
    return base

from .. import db

title = (base.get("lesson") or {}).get("title") or prompt[:40] or "课例复现"
proj = db.create_project(title=title, idea={"prompt": prompt})
hsd_text = preview.get("hsd_preview") or ""
protocol = {
    "dftb_family": preview.get("family"),
    "kind": preview.get("kind"),
    "sk_set": preview.get("sk_set"),
    "prompt": prompt,
    "hsd_preview": hsd_text,
    "hsd_default": hsd_text,
    "playbook_id": (preview.get("playbook") or {}).get("playbook_id"),
    "success_criteria": (preview.get("playbook") or {}).get("success_criteria") or
[],
    "expected_observables": base.get("expected_observables") or [],
    "lesson_id": lesson_id,
    "param_tips": pipeline._param_tips(preview.get("kind") or "",
preview.get("sk_set") or ""),
}
structure: dict = {}
if poscar:
    structure["poscar"] = poscar
if preview.get("mp"):
    structure["mp"] = preview["mp"]
mp_id = (base.get("lesson") or {}).get("mp_id")
if mp_id and "mp" not in structure:
    structure["mp"] = {"material_id": mp_id}
db.update_project(proj["id"], protocol=protocol, phase="protocol",
structure=structure)
preview["param_tips"] = protocol["param_tips"]
preview["hsd_default"] = hsd_text
return {

```

```

        **base,
        "ok": True,
        "recipe_ready": True,
        "project_id": proj["id"],
        "preview": preview,
        "message": f"已为课例「{title}」生成输入文件（HSD），请核对后确认计算。",
    }
}

# ===== FILE: apps/api/dftbneu/services/deepseek.py =====
"""兼容旧导入：转发到 llm。"""

from .llm import DeepSeekError, LLMError, chat, probe, resolve_endpoint

__all__ = ["DeepSeekError", "LLMError", "chat", "probe", "resolve_endpoint"]
# ===== FILE: apps/api/dftbneu/services/deploy.py =====
"""WSL / 本机 DFTB+ 隔离部署与状态探测。"""

from __future__ import annotations

import platform
import shutil
import subprocess
import time
from pathlib import Path
from typing import Any

from ..config import load_user_config, save_user_config, settings
from dftb_engine.local_runner import (
    LocalDftbRunner,
    resolve_wsl_distro,
    run_local,
    run_wsl,
    use_wsl,
    wsl_exe,
)

def _repo_script() -> Path:
    return settings.root / "scripts" / "wsl_deploy_dftb.sh"

def detect_wsl() -> dict[str, Any]:
    """Windows 上检测 WSL2；其它平台视为本机 Linux/macOS。"""
    system = platform.system()
    if system != "Windows":
        return {
            "platform": system,
            "wsl_required": False,
            "wsl_available": False,
            "ready_for_deploy": True,
            "message": "当前为非 Windows 环境，可直接在本机用户目录隔离部署。",
            "admin_commands": [],
            "docs_url": "https://learn.microsoft.com/windows/wsl/install",

```

```

    }
    exe = wsl_exe()
    has_wsl = shutil.which("wsl.exe") is not None or shutil.which("wsl") is not None
    distros: list[str] = []
    default = ""
    version_ok = False
    msg = ""
    if has_wsl:
        try:
            r = subprocess.run([exe, "-l", "-v"], capture_output=True, text=True,
timeout=20)
            text = (r.stdout or "") + (r.stderr or "")
            # wsl -l -v 常为 UTF-16; 再试 --list --verbose
            if not text.strip():
                r2 = subprocess.run([exe, "--list", "--verbose"], capture_output=True,
timeout=20)
                text = (r2.stdout or b"").decode("utf-16-le", errors="replace") + (
                    r2.stderr or b"")
                ).decode("utf-16-le", errors="replace")
            for line in text.splitlines():
                line = line.strip().replace("\x00", "")
                if not line or line.lower().startswith("windows subsystem") or "NAME"
in line.upper():
                    continue
                parts = line.split()
                if not parts:
                    continue
                name = parts[0].lstrip("*").strip()
                if name:
                    distros.append(name)
                    if line.startswith("*") or "*" in parts[0]:
                        default = name
                if "2" in parts[-1:] or "VERSION" not in line.upper() and len(parts) >=
3 and parts[-1] == "2":
                    version_ok = True
            if not default and distros:
                default = distros[0]
            if any("2" in ln.replace("\x00", "") for ln in text.splitlines()):
                version_ok = True
            version_ok = version_ok or bool(distros)
            msg = "已检测到 WSL。" if distros else "已安装 wsl.exe, 但未找到发行版。"
        except Exception as e:
            msg = f"WSL 探测失败: {e}"
    else:
        msg = "未检测到 WSL。点击「部署到本机」将自动启用（需确认管理员权限）。"
    ready = bool(has_wsl and distros)
    return {
        "platform": system,
        "wsl_required": True,
        "wsl_available": has_wsl,
        "wsl_distros": distros,
        "default_distro": default,

```

```

        "wsl2_likely": version_ok,
        "ready_for_deploy": ready,
        "message": msg,
        "admin_commands": [],
        "docs_url": "https://learn.microsoft.com/windows/wsl/install",
    }

def ensure_wsl(*, poll_seconds: int = 90) -> dict[str, Any]:
    """若 WSL/发行版未就绪，提权启动 ``wsl --install -d Ubuntu``，并短时轮询。"""
    info = detect_wsl()
    if not info.get("wsl_required"):
        return {"ok": True, "skipped": True, "wsl": info}
    if info.get("ready_for_deploy"):
        return {"ok": True, "already_ready": True, "wsl": info}

    launched = False
    launch_error = ""
    try:
        import ctypes

# =====
# 以下为源代码连续部分的后部（共后连续约 30 页）
# =====

if __name__ == "__main__":
    raise SystemExit(main())
# ===== FILE: scripts/_bootstrap_hub_deepseek.ps1 =====
$ErrorActionPreference = "Stop"

# 1) force deepseek in .env
$envPath = "D:\cmats-lab\.env"
$raw = Get-Content $envPath -Raw
$raw = [regex]::Replace($raw, "(?m)^\CMATS_LLM_PROVIDER=.*$",
"CMATS_LLM_PROVIDER=deepseek")
[System.IO.File]::WriteAllText($envPath, $raw)

# 2) ensure start script forces provider at end
$launcher = "D:\dftb-neu\scripts\start_classroom_hub_4060.ps1"
$lt = Get-Content $launcher -Raw
if ($lt -notmatch "FORCE DEEPSEEK PROVIDER") {
    $lt = $lt.TrimEnd() + "`r`n`r`n# FORCE DEEPSEEK
PROVIDER`r`n`$env:DFTB_CLASS_LLM_PROVIDER = `\"deepseek`\"`r`n"
    [System.IO.File]::WriteAllText($launcher, $lt)
}

# 3) scheduled task under SYSTEM
$taskName = "DFTB-ClassroomHub"
Unregister-ScheduledTask -TaskName $taskName -Confirm:$false -ErrorAction
SilentlyContinue

```

```

$action = New-ScheduledTaskAction -Execute "powershell.exe" `
    -Argument "-NoProfile -ExecutionPolicy Bypass -File `"$launcher`"" `
    -WorkingDirectory "D:\dftb-neu"
$trigger = New-ScheduledTaskTrigger -AtStartup
$settings = New-ScheduledTaskSettingsSet -AllowStartIfOnBatteries -
DontStopIfGoingOnBatteries `
    -StartWhenAvailable -RestartCount 5 -RestartInterval (New-TimeSpan -Minutes 1) `
    -ExecutionTimeLimit ([TimeSpan]::Zero)
$principal = New-ScheduledTaskPrincipal -UserId "SYSTEM" -LogonType ServiceAccount -
RunLevel Highest
Register-ScheduledTask -TaskName $taskName -Action $action -Trigger $trigger -Settings
$settings -Principal $principal -
Force | Out-Null

Get-NetTCPConnection -LocalPort 8791 -State Listen -ErrorAction SilentlyContinue |
    ForEach-Object { Stop-Process -Id $_.OwningProcess -Force -ErrorAction
SilentlyContinue }
Start-Sleep -Seconds 2
Start-ScheduledTask -TaskName $taskName

$ok = $false
for ($i=1; $i -le 15; $i++) {
    Start-Sleep -Seconds 1
    try {
        $h = Invoke-RestMethod "http://127.0.0.1:8791/health" -TimeoutSec 3
        Write-Host ("ready provider=" + $h.llm_provider + " ds=" + $h.deepseek_key_set)
        $ok = $true
        break
    } catch {}
}
if (-not $ok) { throw "hub not ready" }

& tailscale funnel --https=443 off 2>$null | Out-Null
Start-Sleep -Seconds 1
& tailscale funnel --bg --yes "http://127.0.0.1:8791" | Out-Null
Start-Sleep -Seconds 3
$h2 = Invoke-RestMethod "https://desktop-ibhgp7g.tailcc9705.ts.net/health" -TimeoutSec
20
Write-Host ("public provider=" + $h2.llm_provider)

# wait to ensure not dying immediately
Start-Sleep -Seconds 8
$h3 = Invoke-RestMethod "http://127.0.0.1:8791/health" -TimeoutSec 3
Write-Host ("still_alive provider=" + $h3.llm_provider)
Write-Host "BOOTSTRAP_OK"
# ===== FILE: scripts/_install_hub_task.ps1 =====
# Install/run a Scheduled Task so hub survives SSH disconnect (Windows OpenSSH job kill)
$ErrorActionPreference = "Stop"

# Ensure provider=deepseek in env file
$envPath = "D:\cmats-lab\.env"
if (Test-Path $envPath) {

```

```

$raw = Get-Content $envPath -Raw
if ($raw -match "(?m)^CMATS_LLM_PROVIDER=.*$") {
    $raw = [regex]::Replace($raw, "(?m)^CMATS_LLM_PROVIDER=.*$",
"CMATS_LLM_PROVIDER=deepseek")
} else {
    $raw = $raw.TrimEnd() + "`r`nCMATS_LLM_PROVIDER=deepseek`r`n"
}
[System.IO.File]::WriteAllText($envPath, $raw)
}

$launcher = "D:\dftb-neu\scripts\start_classroom_hub_4060.ps1"
# Patch start script to force deepseek after loading env
$startTxt = Get-Content $launcher -Raw
if ($startTxt -notmatch "FORCE DEEPSEEK PROVIDER") {
    $startTxt = $startTxt -replace \
        'if \((-not \$env:DFTB_CLASS_LLM_PROVIDER\) \{ \$env:DFTB_CLASS_LLM_PROVIDER =
"deepseek" \}', `
        "if (-not \$env:DFTB_CLASS_LLM_PROVIDER) { \$env:DFTB_CLASS_LLM_PROVIDER =
`"deepseek`" }`r`n# FORCE DEEPSEEK PROVID
ER`r`n`$env:DFTB_CLASS_LLM_PROVIDER = `"deepseek`""
    [System.IO.File]::WriteAllText($launcher, $startTxt)
}

$taskName = "DFTB-ClassroomHub"
$action = New-ScheduledTaskAction -Execute "powershell.exe" `
    -Argument "-NoProfile -ExecutionPolicy Bypass -File `"$launcher`"" `
    -WorkingDirectory "D:\dftb-neu"
$trigger = New-ScheduledTaskTrigger -AtStartup
$settings = New-ScheduledTaskSettingsSet -AllowStartIfOnBatteries -
DontStopIfGoingOnBatteries `
    -StartWhenAvailable -RestartCount 3 -RestartInterval (New-TimeSpan -Minutes 1) `
    -ExecutionTimeLimit ([TimeSpan]::Zero)
# SYSTEM so the hub survives SSH disconnect / user session quirks
$principal = New-ScheduledTaskPrincipal -UserId "SYSTEM" -LogonType ServiceAccount -
RunLevel Highest

Unregister-ScheduledTask -TaskName $taskName -Confirm:$false -ErrorAction
SilentlyContinue
Register-ScheduledTask -TaskName $taskName -Action $action -Trigger $trigger -Settings
$settings -Principal $principal -
Force | Out-Null
Write-Host "registered task $taskName"

# Kill any zombie then run task
Get-NetTCPConnection -LocalPort 8791 -State Listen -ErrorAction SilentlyContinue |
    ForEach-Object { Stop-Process -Id $_.OwningProcess -Force -ErrorAction
SilentlyContinue }
Start-Sleep -Seconds 1
Start-ScheduledTask -TaskName $taskName
Start-Sleep -Seconds 6

$h = Invoke-RestMethod "http://127.0.0.1:8791/health" -TimeoutSec 5

```

DFTB-NEU 密度泛函紧束缚计算工作台软件 V1.0

```
Write-Host ("local provider=" + $h.llm_provider + " ds=" + $h.deepseek_key_set)

& tailscale funnel --https=443 off 2>$null | Out-Null
Start-Sleep -Seconds 1
& tailscale funnel --bg --yes "http://127.0.0.1:8791" | Out-Null
Start-Sleep -Seconds 2
$h2 = Invoke-RestMethod "https://desktop-ibhgp7g.tailcc9705.ts.net/health" -TimeoutSec
20
Write-Host ("public provider=" + $h2.llm_provider)

# Funnel keeper task (optional lightweight)
$funnelLauncher = "D:\dftb-neu\scripts\keep_hub_funnel.ps1"
if (Test-Path $funnelLauncher) {
    $ft = "DFTB-HubFunnelKeep"
    $faction = New-ScheduledTaskAction -Execute "powershell.exe" `
        -Argument "-NoProfile -ExecutionPolicy Bypass -File `"$funnelLauncher`"" `
        -WorkingDirectory "D:\dftb-neu"
    # every 5 minutes
    $ftrig = New-ScheduledTaskTrigger -Once -At (Get-Date).Date -RepetitionInterval (New-
Timespan -Minutes 5) -RepetitionD
uration ([Timespan]::MaxValue)
    Unregister-ScheduledTask -TaskName $ft -Confirm:$false -ErrorAction SilentlyContinue
    Register-ScheduledTask -TaskName $ft -Action $faction -Trigger $ftrig -Settings
$settings -Principal $principal -Force
    | Out-Null
    Write-Host "registered funnel keeper $ft"
}

Write-Host "DONE"
# ===== FILE: scripts/_revive_hub.ps1 =====
$ErrorActionPreference = "Continue"
Write-Host "=== err log tail ==="
if (Test-Path "D:\dftb-neu\data\classroom_hub.err.log") {
    Get-Content "D:\dftb-neu\data\classroom_hub.err.log" -Tail 40
}
Write-Host "=== out log tail ==="
if (Test-Path "D:\dftb-neu\data\classroom_hub.out.log") {
    Get-Content "D:\dftb-neu\data\classroom_hub.out.log" -Tail 20
}

Write-Host "=== start hub ==="
powershell -NoProfile -ExecutionPolicy Bypass -File "D:\dftb-
neu\scripts\start_classroom_hub_4060.ps1"
Start-Sleep -Seconds 5
Get-NetTCPConnection -LocalPort 8791 -State Listen -ErrorAction SilentlyContinue |
    Format-Table OwningProcess,State -AutoSize
try {
    $h = Invoke-RestMethod "http://127.0.0.1:8791/health" -TimeoutSec 5
    Write-Host ("local health provider=" + $h.llm_provider + " ds=" + $h.deepseek_key_set)
} catch {
    Write-Host ("local health FAIL: " + $_.Exception.Message)
    Write-Host "=== err after start ==="
```

```

Get-Content "D:\dftb-neu\data\classroom_hub.err.log" -Tail 50 -ErrorAction
SilentlyContinue
}

Write-Host "=== reset funnel ==="
& tailscale funnel --https=443 off 2>$null
Start-Sleep -Seconds 1
& tailscale funnel --bg --yes "http://127.0.0.1:8791"
Start-Sleep -Seconds 3
try {
    $h2 = Invoke-RestMethod "https://desktop-ibhgp7g.tailcc9705.ts.net/health" -
TimeoutSec 20
    Write-Host ("public health provider=" + $h2.llm_provider)
} catch {
    Write-Host ("public health FAIL: " + $_.Exception.Message)
}
# ===== FILE: scripts/_start_hub_keep.ps1 =====
# Robust hub start on 4060: keep process alive, set provider=deepseek, restore funnel
$ErrorActionPreference = "Continue"

# Ensure provider
$envPath = "D:\cmats-lab\.env"
if (Test-Path $envPath) {
    $raw = Get-Content $envPath -Raw
    if ($raw -match "(?m)^\CMATS_LLM_PROVIDER=.*$") {
        $raw = [regex]::Replace($raw, "(?m)^\CMATS_LLM_PROVIDER=.*$",
"CMATS_LLM_PROVIDER=deepseek")
    } else {
        $raw = $raw.TrimEnd() + "`r`nCMATS_LLM_PROVIDER=deepseek`r`n"
    }
    [System.IO.File]::WriteAllText($envPath, $raw)
}

# Kill old listeners
Get-NetTCPConnection -LocalPort 8791 -State Listen -ErrorAction SilentlyContinue |
    ForEach-Object { Stop-Process -Id $_.OwningProcess -Force -ErrorAction
SilentlyContinue }
Get-CimInstance Win32_Process -Filter "Name='python.exe'" -ErrorAction SilentlyContinue |
    Where-Object { $_.CommandLine -and $_.CommandLine -match "classroom_hub|uvicorn" } |
    ForEach-Object { Stop-Process -Id $_.ProcessId -Force -ErrorAction SilentlyContinue }
Start-Sleep -Seconds 2

$env:DFTB_CLASS_HOST = "0.0.0.0"
$env:DFTB_CLASS_PORT = "8791"
$env:DFTB_CLASS_DATA = "D:\dftb-neu\data\classroom"
$env:DFTB_CLASS_TEACHER_PASSWORD = "z1303@"
$env:DFTB_CLASS_ADMIN_SECRET = "z1303@"
$env:DFTB_CLASS_TOKEN_SECRET = "z1303@"
$env:PYTHONPATH = "D:\dftb-neu\apps"
$env:DFTB_CLASS_LLM_PROVIDER = "deepseek"

```

```

function Import-HubEnvFile([string]$path) {
    if (-not (Test-Path $path)) { return }
    Get-Content $path | ForEach-Object {
        $line = $_.Trim()
        if (-not $line -or $line.StartsWith("#")) { return }
        if ($line -match "^(?:export\s+)?([A-Za-z_][A-Za-z0-9_]*)=(.*)$") {
            Set-Item -Path ("Env:" + $Matches[1]) -Value
($Matches[2].Trim().Trim(' ').Trim('"'))
        }
    }
}

Import-HubEnvFile "D:\dftb-neu\data\classroom\secrets.env"
if (Test-Path $envPath) {
    Get-Content $envPath | ForEach-Object {
        if ($_ -match "^CMATS_DEEPSEEK_API_KEY=(.*)$") { $env:DFTB_CLASS_DEEPSEEK_API_KEY =
$Matches[1].Trim() }
        if ($_ -match "^CMATS_LLM_API_KEY=(.*)$") { $env:DFTB_CLASS_LLM_API_KEY =
$Matches[1].Trim() }
        if ($_ -match "^CMATS_MP_API_KEY=(.*)$") { $env:DFTB_CLASS_MP_API_KEY =
$Matches[1].Trim() }
        if ($_ -match "^CMATS_LLM_PROVIDER=(.*)$") { $env:DFTB_CLASS_LLM_PROVIDER =
$Matches[1].Trim() }
        if ($_ -match "^CMATS_DEEPSEEK_BASE_URL=(.*)$") { $env:DFTB_CLASS_DEEPSEEK_BASE_URL =
$Matches[1].Trim() }
        if ($_ -match "^CMATS_DEEPSEEK_MODEL=(.*)$") { $env:DFTB_CLASS_DEEPSEEK_MODEL =
$Matches[1].Trim() }
        if ($_ -match "^CMATS_LLM_BASE_URL=(.*)$") { $env:DFTB_CLASS_LLM_BASE_URL =
$Matches[1].Trim() }
        if ($_ -match "^CMATS_LLM_MODEL=(.*)$") { $env:DFTB_CLASS_LLM_MODEL =
$Matches[1].Trim() }
    }
}

# Force deepseek regardless of file order
$env:DFTB_CLASS_LLM_PROVIDER = "deepseek"

$py = "D:\cmats-lab\.venv\Scripts\python.exe"
if (-not (Test-Path $py)) { $py = "python" }
$outLog = "D:\dftb-neu\data\classroom_hub.out.log"
$errLog = "D:\dftb-neu\data\classroom_hub.err.log"
"" | Set-Content $outLog
"" | Set-Content $errLog

# Use cmd start to detach from SSH session lifetime
$args = "-NoProfile -ExecutionPolicy Bypass -Command `"`& '$py' -m uvicorn
classroom_hub.main:app --app-dir 'D:\dftb-neu\app
pps' --host 0.0.0.0 --port 8791 *`> '$errLog'`""
$ps = Start-Process -FilePath "powershell.exe" -ArgumentList $args -WorkingDirectory
"D:\dftb-neu" `
    -WindowStyle Hidden -PassThru
"wrapper_pid=$( $ps.Id )" | Set-Content "D:\dftb-neu\data\classroom_hub.pid.txt"

$ok = $false

```

```

for ($i = 1; $i -le 10; $i++) {
    Start-Sleep -Seconds 1
    try {
        $h = Invoke-RestMethod "http://127.0.0.1:8791/health" -TimeoutSec 3
        Write-Host ("ready provider=" + $h.llm_provider + " ds=" + $h.deepseek_key_set + "
qw=" + $h.qwen_key_set)
        $ok = $true
        break
    } catch {}
}
if (-not $ok) {
    Write-Host "hub failed to become ready"
    Get-Content $errLog -Tail 40
    exit 1
}

# Funnel
& tailscale funnel --https=443 off 2>$null | Out-Null
Start-Sleep -Seconds 1
& tailscale funnel --bg --yes "http://127.0.0.1:8791" | Out-Null
Start-Sleep -Seconds 2

# Stability check
Start-Sleep -Seconds 5
try {
    $h2 = Invoke-RestMethod "http://127.0.0.1:8791/health" -TimeoutSec 3
    Write-Host ("stable local provider=" + $h2.llm_provider)
} catch {
    Write-Host "UNSTABLE: died within 5s"
    Get-Content $errLog -Tail 40
    exit 2
}
try {
    $h3 = Invoke-RestMethod "https://desktop-ibhgp7g.tailcc9705.ts.net/health" -
TimeoutSec 20
    Write-Host ("public provider=" + $h3.llm_provider)
} catch {
    Write-Host ("public FAIL: " + $_.Exception.Message)
}

# Direct deepseek probe via hub chat requires token; probe upstream instead
$key = $env:DFTB_CLASS_DEEPSEEK_API_KEY
$model = if ($env:DFTB_CLASS_DEEPSEEK_MODEL) { $env:DFTB_CLASS_DEEPSEEK_MODEL } else
{ "deepseek-v4-flash" }
$payload = (@{
    model = $model
    messages = @(@{ role = "user"; content = "Reply with exactly: OK" })
    temperature = 0.0
    max_tokens = 64
} | ConvertTo-Json -Depth 5)
$r = Invoke-WebRequest -Uri "https://api.deepseek.com/chat/completions" -Method POST -
TimeoutSec 60 `

```

```

-headers @{ Authorization = ("Bearer " + $key); "Content-Type" = "application/json" }
,

-Body $payload -UseBasicParsing
$ج = $ر.Content | ConvertFrom-Json
Write-Host ("deepseek_direct status=" + $ر.StatusCode + " content=" +
[string]$ج.choices[0].message.content)
# ===== FILE: scripts/_switch_hub_to_deepseek.ps1 =====
# Run on hub host: set provider=deepseek, restart hub, probe chat
$ErrorActionPreference = "Stop"
$envPath = "D:\cmats-lab\.env"
if (-not (Test-Path $envPath)) { throw "missing $envPath" }

$raw = Get-Content $envPath -Raw
if ($raw -match "(?m)^\CMATS_LLM_PROVIDER=.*$") {
    $raw = [regex]::Replace($raw, "(?m)^\CMATS_LLM_PROVIDER=.*$",
"CMATS_LLM_PROVIDER=deepseek")
} else {
    $raw = $raw.TrimEnd() + "`r`nCMATS_LLM_PROVIDER=deepseek`r`n"
}
[System.IO.File]::WriteAllText($envPath, $raw)
Write-Host "set CMATS_LLM_PROVIDER=deepseek"

$sec = "D:\dftb-neu\data\classroom\secrets.env"
if (Test-Path $sec) {
    $sraw = Get-Content $sec -Raw
    if ($sraw -match "(?m)^\DFTB_CLASS_LLM_PROVIDER=.*$") {
        $sraw = [regex]::Replace($sraw, "(?m)^\DFTB_CLASS_LLM_PROVIDER=.*$",
"DFTB_CLASS_LLM_PROVIDER=deepseek")
    } elseif ($sraw -match "(?m)^\CMATS_LLM_PROVIDER=.*$") {
        $sraw = [regex]::Replace($sraw, "(?m)^\CMATS_LLM_PROVIDER=.*$",
"CMATS_LLM_PROVIDER=deepseek")
    } else {
        $sraw = $sraw.TrimEnd() + "`r`nDFTB_CLASS_LLM_PROVIDER=deepseek`r`n"
    }
    [System.IO.File]::WriteAllText($sec, $sraw)
    Write-Host "updated secrets.env"
}

Write-Host "restarting hub..."
powershell -NoProfile -ExecutionPolicy Bypass -File "D:\dftb-
neu\scripts\start_classroom_hub_4060.ps1"
Start-Sleep -Seconds 4
$ه = Invoke-RestMethod "http://127.0.0.1:8791/health" -TimeoutSec 5
Write-Host ("health provider=" + $ه.llm_provider + " ds=" + $ه.deepseek_key_set + "
qw=" + $ه.qwen_key_set)

function Import-HubEnvFile([string]$path) {
    if (-not (Test-Path $path)) { return }
    Get-Content $path | ForEach-Object {
        $line = $_.Trim()
        if (-not $line -or $line.StartsWith("#")) { return }
        if ($line -match "^(?:export\s+)?([A-Za-z_][A-Za-z0-9_]*)=(.*)$") {

```

```

        Set-Item -Path ("Env:" + $Matches[1]) -Value
($Matches[2].Trim().Trim(' ').Trim(''))
    }
}
Import-HubEnvFile $sec
Get-Content $envPath | ForEach-Object {
    if ($_ -match "^CMATS_DEEPSEEK_API_KEY=(.*)$") { $env:DFTB_CLASS_DEEPSEEK_API_KEY =
$Matches[1].Trim() }
    if ($_ -match "^CMATS_DEEPSEEK_BASE_URL=(.*)$") { $env:DFTB_CLASS_DEEPSEEK_BASE_URL =
$Matches[1].Trim() }
    if ($_ -match "^CMATS_DEEPSEEK_MODEL=(.*)$") { $env:DFTB_CLASS_DEEPSEEK_MODEL =
$Matches[1].Trim() }
}
$key = $env:DFTB_CLASS_DEEPSEEK_API_KEY
$base = if ($env:DFTB_CLASS_DEEPSEEK_BASE_URL)
{ $env:DFTB_CLASS_DEEPSEEK_BASE_URL.TrimEnd("/") } else { "https://api.de
eepseek.com" }
$model = if ($env:DFTB_CLASS_DEEPSEEK_MODEL) { $env:DFTB_CLASS_DEEPSEEK_MODEL } else
{ "deepseek-chat" }
$url = $base + "/chat/completions"
$payloadObj = @{
    model = $model
    messages = @(
        @{ role = "system"; content = "Be concise." },
        @{ role = "user"; content = "Reply with exactly: OK" }
    )
    temperature = 0.0
    max_tokens = 16
}
$payload = $payloadObj | ConvertTo-Json -Depth 5
Write-Host ("probe url=" + $url + " model=" + $model)
$r = Invoke-WebRequest -Uri $url -Method POST -TimeoutSec 60 `
    -Headers @{ Authorization = ("Bearer " + $key); "Content-Type" = "application/json" }
    -Body $payload -UseBasicParsing
Write-Host ("status=" + $r.StatusCode)
$j = $r.Content | ConvertFrom-Json
$c = [string]$j.choices[0].message.content
Write-Host ("content_len=" + $c.Length)
Write-Host ("content=" + $c)
Write-Host ("finish_reason=" + [string]$j.choices[0].finish_reason)
# ===== FILE: scripts/create_classroom.sh =====
#!/usr/bin/env bash
# 教师：设置本堂学生短密码（教师永久密码默认 z1303@）
set -euo pipefail
HUB_URL="{DFTB_CLASS_PUBLIC_URL:-http://127.0.0.1:8791}"
TEACHER="{DFTB_CLASS_TEACHER_PASSWORD:-z1303@}"
STUDENT_CODE="{1:?用法: $0 <本堂学生短密码> [有效天数]}"
DAYS="{2:-30}"
curl -fsS -X POST "$HUB_URL/teacher/gate" \
    -H "Content-Type: application/json" \

```

```

-H "X-Teacher-Password: $TEACHER" \
-d "{\"student_password\":\"$STUDENT_CODE\",\"session_days\":$DAYS}" | python3 -m
json.tool
echo
echo "请告诉学生本堂口令: $STUDENT_CODE"
echo "学生只需在软件「进入」页输入该口令。"
# ===== FILE: scripts/fix-win-ssh.ps1 =====
# DFTB: one-shot OpenSSH fix for Windows laptop (admin PowerShell)
$ErrorActionPreference = "Continue"
Write-Host "=== DFTB OpenSSH fix ==="

# 1) Locate or reinstall OpenSSH
$sshd = Get-Command sshd.exe -ErrorAction SilentlyContinue
$dirs = @(
    "C:\Program Files\OpenSSH",
    "C:\Program Files\OpenSSH-Win64",
    "C:\Program Files\OpenSSH\OpenSSH-Win64",
    "$env:ProgramFiles\OpenSSH",
    "$env:ProgramFiles\OpenSSH-Win64"
)
$sshDir = $dirs | Where-Object { Test-Path (Join-Path $_ "sshd.exe") } | Select-Object
-First 1

if (-not $sshDir) {
    Write-Host "OpenSSH binaries missing; reinstalling Preview via winget..."
    winget install --id Microsoft.OpenSSH.Preview -e --accept-source-agreements --accept-
package-agreements --disable-inte
ractivity
    Start-Sleep -Seconds 2
    $sshDir = $dirs | Where-Object { Test-Path (Join-Path $_ "sshd.exe") } | Select-
Object -First 1
}

if (-not $sshDir) {
    # fallback: download zip
    Write-Host "winget path failed; downloading OpenSSH zip..."
    $zip = "$env:TEMP\OpenSSH-Win64.zip"
    $url = "https://github.com/PowerShell/Win32-OpenSSH/releases/download/v9.8.3.0p2-
Preview/OpenSSH-Win64.zip"
    [Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12
    Invoke-WebRequest -Uri $url -OutFile $zip
    $dest = "C:\Program Files\OpenSSH-Win64"
    New-Item -ItemType Directory -Force -Path $dest | Out-Null
    Expand-Archive -Path $zip -DestinationPath $dest -Force
    if (Test-Path "$dest\OpenSSH-Win64\sshd.exe") { $sshDir = "$dest\OpenSSH-Win64" }
    elseif (Test-Path "$dest\sshd.exe") { $sshDir = $dest }
}

if (-not $sshDir -or -not (Test-Path "$sshDir\sshd.exe")) {
    Write-Host "FATAL: sshd.exe not found"
    exit 1
}

```

```

Write-Host "OpenSSH dir: $oshDir"

# 2) Register service if missing
$svc = Get-Service -Name sshd -ErrorAction SilentlyContinue
if (-not $svc) {
    $svc = Get-Service | Where-Object { $_.Name -match 'ssh' -or $_.DisplayName -match
'OpenSSH.*Server' } | Select-Object
    -First 1
}
if (-not $svc) {
    Write-Host "Registering sshd service..."
    if (Test-Path "$oshDir\install-sshd.ps1") {
        & "$oshDir\install-sshd.ps1"
    } else {
        & "$oshDir\sshd.exe" install-sshd 2>$null
        if (-not (Get-Service sshd -ErrorAction SilentlyContinue)) {
            New-Service -Name sshd -BinaryPathName "`"$oshDir\sshd.exe`"" -DisplayName
"OpenSSH SSH Server" -StartupType Autom
atic | Out-Null
        }
    }
}

# Ensure PATH contains OpenSSH for this session
$env:Path = "$oshDir;" + $env:Path

# 3) Firewall
try {
    New-NetFirewallRule -Name "OpenSSH-Server-In-TCP" -DisplayName "OpenSSH Server
(sshd)" -Enabled True -Direction Inbound
    -Protocol TCP -Action Allow -LocalPort 22 -ErrorAction Stop | Out-Null
} catch {
    # already exists is fine
}

# 4) Admin authorized_keys (required for Administrators group users)
New-Item -ItemType Directory -Force -Path "$env:ProgramData\ssh" | Out-Null
$pub = "ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQAD7t4UAsUTY7f2nZ1FBJGeuqqU9KWl3BUQ85H3kthc/ouBxiuVEX81Wnmi
ISz90Fm0arCKvcWz4
l/coAgCBz4RibF50kOSnjcvn4B2aJy6w/B1FqZ/lwzqfEDLsdMKV7IhmQpDGZ1/PgLe1OQd0AgjcN3YH5tUFRtd
b+Kp5W2kb4CrJWFxDfCAeDIjPzw6JAtON
pP5OQR96AIaDhbrth0iEV9sjv+r7BWb/ZXhmNNivYJGL3/mvtq8ty29ZyCN9AJ8GVubP/kyUnNLXGgmkl/By3/g
vnVaViV76tnuJUxhGcw7BNoc6W0VKeU6j
buvK4GtuUvRl7lB4ROdCkLZ+o4WhC68jQsYOM+vJYWm5+zaZNwdyXfaKnqFy4jDumtFWsTBbR830suoN3w+H/ee
U/xD5WZSx3ZNN6FQOeblHsBxtnugp9C9P
Joyk28IxiyFyqZrrKav5GKc275EnafPRv1v5r5BZdVUGl69LJ+hnhYUSQEU6fNAtAGraczKJV2DQigPXxfkOZJg
R8vZ5kZkntVxwwLgbjzXoiz5aSumm90KO
x3Jbc8ofIC9Wh9x/g/fHgF8eo9UdIx7fU1qr2Z/wyNbXSLlkgjGeeWV2z1Ie8YIAxE+dAJaiiN94ppqza64B80Y
iFir9QEnpDTryho4TCUq90qXxQzsFTI2g
C/UEIoSIwQ== mu@mudeMac-mini.local"
$akAdmin = "$env:ProgramData\ssh\administrators_authorized_keys"

```

```

Set-Content -Path $akAdmin -Value $pub -Encoding ascii
cmd /c "icacls `"$akAdmin`" /inheritance:r" | Out-Null
cmd /c "icacls `"$akAdmin`" /grant SYSTEM:(F)" | Out-Null
cmd /c "icacls `"$akAdmin`" /grant *S-1-5-32-544:(F)" | Out-Null

# Also user key (non-admin path)
New-Item -ItemType Directory -Force -Path "$env:USERPROFILE\.ssh" | Out-Null
$akUser = "$env:USERPROFILE\.ssh\authorized_keys"
Set-Content -Path $akUser -Value $pub -Encoding ascii
cmd /c "icacls `"$env:USERPROFILE\.ssh`" /inheritance:r" | Out-Null
cmd /c "icacls `"$env:USERPROFILE\.ssh`" /grant `"$env:USERNAME`:(F)`" | Out-Null
cmd /c "icacls `"$akUser`" /inheritance:r" | Out-Null
cmd /c "icacls `"$akUser`" /grant `"$env:USERNAME`:(R)`" | Out-Null

# 5) sshd_config essentials
$cfgCandidates = @(
    "$env:ProgramData\ssh\sshd_config",
    "$oshDir\sshd_config_default",
    "$oshDir\sshd_config"
)
$cfg = "$env:ProgramData\ssh\sshd_config"
if (-not (Test-Path $cfg)) {
    $src = $cfgCandidates | Where-Object { Test-Path $_ } | Select-Object -First 1
    if ($src -and $src -ne $cfg) { Copy-Item $src $cfg -Force }
}
if (Test-Path $cfg) {
    $txt = Get-Content $cfg -Raw
    if ($txt -notmatch "(?m)^PubkeyAuthentication\s+yes") {
        Add-Content $cfg "`nPubkeyAuthentication yes`nPasswordAuthentication yes`n"
    }
}

# 6) Start service
$svcName = "sshd"
if (-not (Get-Service -Name sshd -ErrorAction SilentlyContinue)) {
    $alt = Get-Service | Where-Object { $_.DisplayName -match 'OpenSSH.*Server' } |
    Select-Object -First 1
    if ($alt) { $svcName = $alt.Name }
}
Write-Host "Service name: $svcName"
try { Set-Service -Name $svcName -StartupType Automatic } catch {}
try { Start-Service -Name $svcName -ErrorAction Stop } catch {
    Write-Host "Start-Service failed: $($_.Exception.Message)"
    # last resort: run sshd in background once
    Start-Process -FilePath "$oshDir\sshd.exe" -ArgumentList "-D" -WindowStyle Hidden
}
Start-Sleep -Seconds 2

Write-Host "--- status ---"
Get-Service -Name $svcName -ErrorAction SilentlyContinue | Format-List
Name,Status,StartType

```

```

Get-NetTCPConnection -LocalPort 22 -State Listen -ErrorAction SilentlyContinue |
Select-Object LocalAddress,LocalPort
Write-Host "USER=$env:USERNAME"
Write-Host "WHOAMI=$(whoami)"
Write-Host "=== DONE: tell agent OK ==="
# ===== FILE: scripts/hub_public_proxy.py =====
"""Local TCP reverse proxy: 127.0.0.1:18791 -> classroom hub (Tailscale LAN)."""
import asyncio
import os

UPSTREAM_HOST = os.environ.get("DFTB_HUB_UPSTREAM_HOST", "100.127.118.69")
UPSTREAM_PORT = int(os.environ.get("DFTB_HUB_UPSTREAM_PORT", "8791"))
LISTEN_HOST = os.environ.get("DFTB_HUB_PROXY_HOST", "127.0.0.1")
LISTEN_PORT = int(os.environ.get("DFTB_HUB_PROXY_PORT", "18791"))

async def pipe(reader, writer):
    try:
        while True:
            data = await reader.read(65536)
            if not data:
                break
            writer.write(data)
            await writer.drain()
    except Exception:
        pass
    finally:
        try:
            writer.close()
            await writer.wait_closed()
        except Exception:
            pass

async def handle(local_reader, local_writer):
    try:
        remote_reader, remote_writer = await asyncio.open_connection(UPSTREAM_HOST,
UPSTREAM_PORT)
    except Exception as e:
        local_writer.close()
        print("upstream fail", e, flush=True)
        return
    await asyncio.gather(
        pipe(local_reader, remote_writer),
        pipe(remote_reader, local_writer),
    )

async def main():
    server = await asyncio.start_server(handle, LISTEN_HOST, LISTEN_PORT)
    print(f"proxy {LISTEN_HOST}:{LISTEN_PORT} -> {UPSTREAM_HOST}:{UPSTREAM_PORT}",
flush=True)
    async with server:
        await server.serve_forever()

```

```

if __name__ == "__main__":
    asyncio.run(main())

# ===== FILE: scripts/install_hub_public_tunnel_autostart.ps1 =====
# 注册“登录时自动启动”课堂公网隧道守护
# powershell -NoProfile -ExecutionPolicy Bypass -File
scripts\install_hub_public_tunnel_autostart.ps1
$ErrorActionPreference = "Stop"
$Root = Split-Path -Parent $PSScriptRoot
$Keep = Join-Path $Root "scripts\keep_hub_public_tunnel.ps1"
$Start = Join-Path $Root "scripts\start_hub_public_tunnel.ps1"
if (-not (Test-Path $Keep)) { throw "missing $Keep" }

$TaskName = "DFTB-Classroom-Hub-Public-Tunnel"
$arg = "-NoProfile -ExecutionPolicy Bypass -WindowStyle Hidden -File `"$Keep`""
$action = New-ScheduledTaskAction -Execute "powershell.exe" -Argument $arg -
WorkingDirectory $Root
$trigger = New-ScheduledTaskTrigger -AtLogOn -User $env:USERNAME
$settings = New-ScheduledTaskSettingsSet -AllowStartIfOnBatteries -
DontStopIfGoingOnBatteries -StartWhenAvailable -Resta
rtCount 3 -RestartInterval (New-TimeSpan -Minutes 1)
$principal = New-ScheduledTaskPrincipal -UserId $env:USERNAME -LogonType Interactive -
RunLevel Limited

Unregister-ScheduledTask -TaskName $TaskName -Confirm:$false -ErrorAction
SilentlyContinue
Register-ScheduledTask -TaskName $TaskName -Action $action -Trigger $trigger -Settings
$settings -Principal $principal -
Force | Out-Null

# 立刻先起一次隧道 + 守护（避免等到下次登录）
Start-Process powershell.exe -ArgumentList "-NoProfile","-ExecutionPolicy","Bypass","-
File",$Start -WindowStyle Hidden |
Out-Null
Start-Sleep -Seconds 2
# 若守护未在跑则启动
$alive = Get-CimInstance Win32_Process -EA SilentlyContinue |
Where-Object { $_.CommandLine -and $_.CommandLine -match
"keep_hub_public_tunnel\ps1" }
if (-not $alive) {
    Start-Process powershell.exe -ArgumentList "-NoProfile","-
ExecutionPolicy","Bypass","-WindowStyle","Hidden","-File",$K
eep -WorkingDirectory $Root | Out-Null
}

Write-Output "scheduled_task=$TaskName"
Get-ScheduledTask -TaskName $TaskName | Format-List TaskName, State
Write-Output "keep_script=$Keep"
# ===== FILE: scripts/keep_hub_funnel.ps1 =====
# Keep Tailscale Funnel public classroom hub alive (stable HTTPS, no localhost.run).
# Prefer running on 4060 (hub host). Can also run from laptop via SSH to 4060.
$ErrorActionPreference = "Continue"
$Root = Split-Path -Parent $PSScriptRoot

```

```

$UrlFile = Join-Path $Root "data\classroom\public_hub_url.txt"
$LogFile = Join-Path $env:TEMP "dftb-hub-funnel-keep.log"
$FunnelUrl = if ($env:DFTB_FUNNEL_HUB_URL) { $env:DFTB_FUNNEL_HUB_URL.TrimEnd("/") }
else { "https://desktop-ibhgp7g.tai
lcc9705.ts.net" }
$HubPort = if ($env:DFTB_CLASS_PORT) { [int]$env:DFTB_CLASS_PORT } else { 8791 }
$RemoteHost = if ($env:DFTB_FUNNEL_SSH_HOST) { $env:DFTB_FUNNEL_SSH_HOST } else
{ "4060" }
$IntervalSec = if ($env:DFTB_FUNNEL_KEEP_INTERVAL)
{ [int]$env:DFTB_FUNNEL_KEEP_INTERVAL } else { 45 }

function Write-Log([string]$msg) {
    $line = "$(Get-Date -Format 'yyyy-MM-dd HH:mm:ss') $msg"
    Add-Content -Path $LogFile -Value $line -Encoding utf8
}

function Test-Url([string]$url) {
    if (-not $url) { return $false }
    try {
        # Prefer Python/certifi path when available; fall back to Invoke-WebRequest.
        $py = Join-Path $Root ".venv\Scripts\python.exe"
        if (Test-Path $py) {
            $code = @"
import httpx, certifi, sys
u=sys.argv[1]
try:
    r=httpx.get(u+'/health', timeout=20, verify=certifi.where(), trust_env=False)
    sys.exit(0 if r.status_code==200 else 1)
except Exception:
    sys.exit(1)
"@
            & $py -c $code $url | Out-Null
            return ($LASTEXITCODE -eq 0)
        }
        $r = Invoke-WebRequest "$url/health" -UseBasicParsing -TimeoutSec 15
        return ($r.StatusCode -eq 200)
    } catch {
        return $false
    }
}

function Test-LocalHub {
    try {
        $r = Invoke-WebRequest "http://127.0.0.1:$HubPort/health" -UseBasicParsing -
TimeoutSec 3
        return ($r.StatusCode -eq 200)
    } catch { return $false }
}

function Ensure-LocalFunnel {
    if (-not (Test-LocalHub)) {
        Write-Log "local hub down on :$HubPort"
    }
}

```

```

        return $false
    }
    $status = & tailscale funnel status 2>&1 | Out-String
    if ($status -match [regex]::Escape(($FunnelUrl -replace '^https://','')) -and $status -match "Funnel on") {
        return $true
    }
    & tailscale funnel --bg --yes "http://127.0.0.1:$HubPort" 2>&1 | Out-Null
    Start-Sleep -Seconds 2
    return $true
}

function Ensure-RemoteFunnel {
    $cmd = @"
`$ErrorActionPreference='Continue'
try { Invoke-WebRequest http://127.0.0.1:$HubPort/health -UseBasicParsing -TimeoutSec 3
| Out-Null } catch { exit 2 }
`$st = tailscale funnel status 2>&1 | Out-String
if (`$st -notmatch 'Funnel on' -or `$st -notmatch '8791') {
    tailscale funnel --bg --yes http://127.0.0.1:$HubPort | Out-Null
}
exit 0
"@
    $b64 = [Convert]::ToBase64String([Text.Encoding]::Unicode.GetBytes($cmd))
    ssh -o BatchMode=yes -o ConnectTimeout=12 $RemoteHost "powershell -NoProfile -
EncodedCommand $b64" 2>&1 | Out-Null
    return ($LASTEXITCODE -eq 0)
}

New-Item -ItemType Directory -Force -Path (Split-Path $UrlFile) | Out-Null
Set-Content -Path $UrlFile -Value $FunnelUrl -Encoding utf8
Write-Log "keep-funnel start url=$FunnelUrl"

$IsHubHost = Test-LocalHub
while ($true) {
    try {
        if (Test-Url $FunnelUrl) {
            Write-Log "ok $FunnelUrl"
        } else {
            Write-Log "down; repairing"
            if ($IsHubHost -or (Test-LocalHub)) {
                [void](Ensure-LocalFunnel)
            } else {
                [void](Ensure-RemoteFunnel)
            }
        }
        Start-Sleep -Seconds 5
        if (Test-Url $FunnelUrl) { Write-Log "repaired" } else { Write-Log "still down" }
    } catch {
        Write-Log ("err " + $_.Exception.Message)
    }
    Start-Sleep -Seconds $IntervalSec
}

```

```

}
# ===== FILE: scripts/keep_hub_public_tunnel.ps1 =====
# Deprecated: localhost.run keeper retired.
# Use Tailscale Funnel keeper instead.
$erroractionpreference = "Stop"
Write-Warning "keep_hub_public_tunnel.ps1 已停用 (localhost.run)。改走
scripts\\keep_hub_funnel.ps1"
& (Join-Path $PSScriptRoot "keep_hub_funnel.ps1")
# ===== FILE: scripts/make-shortcuts-laptop.ps1 =====
$W = New-Object -ComObject WScript.Shell
$desk = [Environment]::GetFolderPath("Desktop")
$stu = (Get-ChildItem "C:\Users\13919\AppData\Local\Programs\dftb-neu\*.exe" | Where-
Object { $_.Name -notlike "Uninstal
l*" } | Select-Object -First 1).FullName
$tea = (Get-ChildItem "C:\Users\13919\AppData\Local\Programs\dftb-neu-teacher\*.exe" |
Where-Object { $_.Name -notlike "
Uninstall*" } | Select-Object -First 1).FullName
Write-Host "stu=$stu"
Write-Host "tea=$tea"
$s1 = $W.CreateShortcut("$desk\DFTB-Neu.lnk")
$s1.TargetPath = $stu
$s1.WorkingDirectory = Split-Path $stu
$s1.Save()
$s2 = $W.CreateShortcut("$desk\DFTB-Neu-Teacher.lnk")
$s2.TargetPath = $tea
$s2.WorkingDirectory = Split-Path $tea
$s2.Save()
Get-ChildItem "$desk\DFTB*.lnk" | ForEach-Object { Write-Host $_.Name }
Write-Host DONE
# ===== FILE: scripts/pack-win-source.mjs =====
/**
 * 打 Windows 源码包 (可重新打包 Setup, 不含 node_modules / pydeps / release)。
 *   node scripts/pack-win-source.mjs
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { spawnSync } from "node:child_process";

const repoRoot = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
const version =
  JSON.parse(
    fs.readFileSync(
      path.join(repoRoot, "packaging", "windows", "electron", "package.json"),
      "utf8",
    ),
  ).version || "dev";

const outName = `DFTB_Neu_Windows 源码_${version}`;
const desktop = process.env.USERPROFILE
  ? path.join(process.env.USERPROFILE, "Desktop")
  : repoRoot;

```

```

const staging = path.join(repoRoot, "packaging", "windows", "release",
  `_src_${version}`);
const zipPath = path.join(desktop, `${outName}.zip`);
const releaseZip = path.join(repoRoot, "packaging", "windows", "release",
  `${outName}.zip`);

const SKIP_DIR_NAMES = new Set([
  ".git",
  ".venv",
  "venv",
  "node_modules",
  "__pycache__",
  ".pytest_cache",
  ".mypy_cache",
  ".cache",
  "dist",
  "release",
  "python-win",
  "pydeps",
  "win-unpacked",
  "payload",
  "payload-student",
  "payload-teacher",
  "installer-build-student",
  "installer-build-teacher",
  "data",
  ".cursor",
  "agent-transcripts",
  "_src_0.3.1",
]);

const SKIP_FILE_GLOBS = [
  /\.pyc$/i,
  /\.log$/i,
  /^\.env$/i,
  /\.DS_Store$/i,
  /^DFTB_Neu_.*\.(exe|zip)$/i,
  /^app\.zip$/i,
  /^cloudflared\.exe$/i,
];

function shouldSkipDir(name) {
  return SKIP_DIR_NAMES.has(name);
}

function shouldSkipFile(name) {
  return SKIP_FILE_GLOBS.some((re) => re.test(name));
}

function rmrf(p) {
  fs.rmSync(p, { recursive: true, force: true });
}

```

```
function copyTree(src, dest) {
  fs.mkdirSync(dest, { recursive: true });
  for (const ent of fs.readdirSync(src, { withFileTypes: true })) {
    if (ent.name === "." || ent.name === "..") continue;
    const from = path.join(src, ent.name);
    const to = path.join(dest, ent.name);
    if (ent.isDirectory()) {
      if (shouldSkipDir(ent.name)) continue;
      // packaging/windows/electron/dist 等已由 SKIP_DIR_NAMES 覆盖
      copyTree(from, to);
      continue;
    }
    if (ent.isSymbolicLink()) continue;
    if (shouldSkipFile(ent.name)) continue;
    fs.copyFileSync(from, to);
  }
}
```

```
function writeReadme(destRoot) {
  const text = `DFTB-NEU Windows 源码包 ${version}
=====
```

内容: 仓库源码 (apps / engines / templates / scripts / packaging 等),
不含 node_modules、内置 Python、pydeps、已打好的 Setup。

在 Windows 上重新打 Setup:

1. 安装 Node.js LTS、Git (可选)
2. cd packaging\windows
3. npm install --registry=https://registry.npmirror.com
4. 首次需准备内置 Python 与 pydeps (去掉 SKIP_*) :

```
node scripts\pack-win.mjs
```

 若本机已有 packaging\windows\backend\python-win 与 pydeps, 可:

```
set SKIP_WIN_PYTHON=1
set SKIP_PYDEPS=1
node scripts\pack-win.mjs
```
5. 产物在 packaging\windows\release\DFTB_Neu_Setup_\${version}.exe

学生使用请发 Setup, 不要发本源码包。

```
`;
  fs.writeFileSync(path.join(destRoot, "源码说明.txt"), text, "utf8");
}
```

```
console.log("staging →", staging);
rmrf(staging);
const rootOut = path.join(staging, outName);
fs.mkdirSync(rootOut, { recursive: true });
```

// 顶层选择性拷贝 (避免把巨大无关目录带上)

```
const topKeep = [
  "apps",
  "engines",
```

```

"templates",
"scripts",
"docs",
"packaging",
"README.md",
"LICENSE",
".gitignore",
"pyproject.toml",
"requirements.txt",
"requirements-dev.txt",
];
for (const name of topKeep) {
  const from = path.join(repoRoot, name);
  if (!fs.existsSync(from)) continue;
  const to = path.join(rootOut, name);
  const st = fs.statSync(from);
  if (st.isDirectory()) copyTree(from, to);
  else if (!shouldSkipFile(name)) fs.copyFileSync(from, to);
}
writeReadme(rootOut);

// 用内置 python 打 zip (路径友好)
const py =
  process.env.DFTB_NEU_PYTHON ||
  path.join(repoRoot, "packaging", "windows", "backend", "python-win", "python.exe");
// 源码 zip: 保留顶层目录名; 不要求根目录有 exe
const zipPy = `
import sys, zipfile
from pathlib import Path
src, out = Path(sys.argv[1]), Path(sys.argv[2])
if out.exists(): out.unlink()
root_name = src.name
n = 0
with zipfile.ZipFile(out, "w", compression=zipfile.ZIP_DEFLATED, compresslevel=6) as zf:
  for p in sorted(src.rglob("*")):
    if p.is_file():
      zf.write(p, arcname=(root_name + "/" + p.relative_to(src).as_posix()))
      n += 1
print(f"wrote {out} files={n} size_mb={out.stat().st_size/1024/1024:.1f}")
`;
const zipHelper = path.join(staging, "_zip_src.py");
fs.writeFileSync(zipHelper, zipPy, "utf8");
const pyBin = fs.existsSync(py) ? py : "python";
for (const dest of [zipPath, releaseZip]) {
  rmrf(dest);
  console.log("zip →", dest);
  const r = spawnSync(pyBin, [zipHelper, rootOut, dest], { stdio: "inherit" });
  if (r.status !== 0) process.exit(r.status || 1);
}

rmrf(staging);
console.log("DONE");

```

```

console.log(" Desktop:", zipPath);
console.log(" Release:", releaseZip);
# ===== FILE: scripts/restart_classroom_hub_4060.bat =====
@echo off
setlocal
powershell -NoProfile -Command "Get-NetTCPConnection -LocalPort 8791 -State Listen -
ErrorAction SilentlyContinue | ForEa
ch-Object { Stop-Process -Id $_.OwningProcess -Force -ErrorAction SilentlyContinue }"
ping -n 2 127.0.0.1 >nul
if not exist "D:\dftb-neu\data" mkdir "D:\dftb-neu\data"
if not exist "D:\dftb-neu\data\classroom" mkdir "D:\dftb-neu\data\classroom"
schtasks /Delete /TN dftb-classroom-hub /F >nul 2>&1
schtasks /Create /TN dftb-classroom-hub /TR "D:\dftb-
neu\scripts\run_classroom_hub_4060.bat" /SC ONLOGON /RL LIMITED /F
schtasks /Run /TN dftb-classroom-hub
ping -n 8 127.0.0.1 >nul
echo classroom_hub_restarted
endlocal
# ===== FILE: scripts/run_api.sh =====
#!/usr/bin/env bash
# 开发启动本地 API + 课堂 UI
set -euo pipefail
ROOT="$(cd "$(dirname "$0")/.." && pwd)"
export PYTHONPATH="$ROOT/apps/api:$ROOT/engines/dftb_agent${PYTHONPATH:+:$PYTHONPATH}"
cd "$ROOT/apps/api"
exec python3 -m uvicorn dftbneu.main:app --host 127.0.0.1 --port "${DFTB_NEU_PORT:-
8765}" "$@"
# ===== FILE: scripts/run_classroom_hub.sh =====
#!/usr/bin/env bash
# 在 4060 上启动课堂中心（默认 0.0.0.0:8791，供公网/反向代理）
set -euo pipefail
ROOT="$(cd "$(dirname "$0")/.." && pwd)"
export PYTHONPATH="$ROOT/apps${PYTHONPATH:+:$PYTHONPATH}"
cd "$ROOT"

# 教师永久密码默认 z1303@
export DFTB_CLASS_TEACHER_PASSWORD="${DFTB_CLASS_TEACHER_PASSWORD:-z1303@}"
export DFTB_CLASS_ADMIN_SECRET="${DFTB_CLASS_ADMIN_SECRET:-
$DFTB_CLASS_TEACHER_PASSWORD}"
export DFTB_CLASS_TOKEN_SECRET="${DFTB_CLASS_TOKEN_SECRET:-
$DFTB_CLASS_TEACHER_PASSWORD}"

# 可选：从并列 cmats-lab/.env 导入上游 Key（按行解析，避免 source 踩坑）
if [ -f "$ROOT/../cmats-lab/.env" ]; then
    while IFS= read -r line || [ -n "$line" ]; do
        case "$line" in
            CMATS_DEEPSEEK_API_KEY=*|CMATS_LLM_API_KEY=*|CMATS_MP_API_KEY=*|CMATS_LLM_PROVIDER=*)
                export "$line"
            ;;
            ;;
        esac
    done < "$ROOT/../cmats-lab/.env" || true

```

```

export DFTB_CLASS_DEEPSEEK_API_KEY="${DFTB_CLASS_DEEPSEEK_API_KEY:-
${CMATS_DEEPSEEK_API_KEY:-}}"
export DFTB_CLASS_LLM_API_KEY="${DFTB_CLASS_LLM_API_KEY:-${CMATS_LLM_API_KEY:-}}"
export DFTB_CLASS_MP_API_KEY="${DFTB_CLASS_MP_API_KEY:-${CMATS_MP_API_KEY:-}}"
export DFTB_CLASS_LLM_PROVIDER="${DFTB_CLASS_LLM_PROVIDER:-${CMATS_LLM_PROVIDER:-
deepseek}}"
fi

HOST="${DFTB_CLASS_HOST:-0.0.0.0}"
PORT="${DFTB_CLASS_PORT:-8791}"
exec python3 -m uvicorn classroom_hub.main:app --host "$HOST" --port "$PORT"
# ===== FILE: scripts/run_classroom_hub_4060.bat =====
@echo off
cd /d D:\dftb-neu
D:\cmats-lab\.venv\Scripts\python.exe D:\dftb-neu\scripts\run_classroom_hub_4060.py
# ===== FILE: scripts/run_classroom_hub_4060.py =====
"""Start DFTB classroom hub on 4060 (Windows)."""
from __future__ import annotations

import os
import sys
from pathlib import Path

ROOT = Path(r"D:\dftb-neu")
APPS = ROOT / "apps"
DATA = ROOT / "data" / "classroom"
DATA.mkdir(parents=True, exist_ok=True)

os.environ.setdefault("DFTB_CLASS_HOST", "0.0.0.0")
os.environ.setdefault("DFTB_CLASS_PORT", "8791")
os.environ.setdefault("DFTB_CLASS_DATA", str(DATA))
os.environ.setdefault("DFTB_CLASS_TEACHER_PASSWORD", "z1303@")
os.environ.setdefault("DFTB_CLASS_ADMIN_SECRET", "z1303@")
os.environ.setdefault("DFTB_CLASS_TOKEN_SECRET", "z1303@")

env_file = Path(r"D:\cmats-lab\.env")
if env_file.is_file():
    mapping = {
        "CMATS_DEEPSEEK_API_KEY": "DFTB_CLASS_DEEPSEEK_API_KEY",
        "CMATS_LLM_API_KEY": "DFTB_CLASS_LLM_API_KEY",
        "CMATS_MP_API_KEY": "DFTB_CLASS_MP_API_KEY",
        "CMATS_LLM_PROVIDER": "DFTB_CLASS_LLM_PROVIDER",
        "CMATS_DEEPSEEK_BASE_URL": "DFTB_CLASS_DEEPSEEK_BASE_URL",
        "CMATS_DEEPSEEK_MODEL": "DFTB_CLASS_DEEPSEEK_MODEL",
        "CMATS_LLM_BASE_URL": "DFTB_CLASS_LLM_BASE_URL",
        "CMATS_LLM_MODEL": "DFTB_CLASS_LLM_MODEL",
    }
    for line in env_file.read_text(encoding="utf-8", errors="ignore").splitlines():
        line = line.strip()
        if not line or line.startswith("#") or "=" not in line:
            continue
        k, v = line.split("=", 1)

```

```

dst = mapping.get(k.strip())
if dst and v.strip() and dst not in os.environ:
    os.environ[dst] = v.strip()

os.environ.setdefault("DFTB_CLASS_LLM_PROVIDER", "deepseek")

sys.path.insert(0, str(APPS))

import uvicorn

if __name__ == "__main__":
    host = os.environ.get("DFTB_CLASS_HOST", "0.0.0.0")
    port = int(os.environ.get("DFTB_CLASS_PORT", "8791"))
    uvicorn.run("classroom_hub.main:app", host=host, port=port, app_dir=str(APPS))
# ===== FILE: scripts/start_classroom_hub_4060.ps1 =====
$ErrorActionPreference = "Continue"

Get-NetTCPConnection -LocalPort 8791 -State Listen -ErrorAction SilentlyContinue |
    ForEach-Object { Stop-Process -Id $_.OwningProcess -Force -ErrorAction
SilentlyContinue }
Start-Sleep -Seconds 1

$env:DFTB_CLASS_HOST = "0.0.0.0"
$env:DFTB_CLASS_PORT = "8791"
$env:DFTB_CLASS_DATA = "D:\dftb-neu\data\classroom"
$env:DFTB_CLASS_TEACHER_PASSWORD = "z1303@"
$env:DFTB_CLASS_ADMIN_SECRET = "z1303@"
$env:DFTB_CLASS_TOKEN_SECRET = "z1303@"
$env:PYTHONPATH = "D:\dftb-neu\apps"

function Import-HubEnvFile([string]$path) {
    if (-not (Test-Path $path)) { return }
    Get-Content $path | ForEach-Object {
        $line = $_.Trim()
        if (-not $line -or $line.StartsWith("#")) { return }
        if ($line -match "^(?:export\s+)?([A-Za-z_][A-Za-z0-9_]*)(.*)$" ) {
            $k = $Matches[1]
            $v = $Matches[2].Trim().Trim('\'').Trim('\"')
            Set-Item -Path "Env:$k" -Value $v
        }
    }
}

Import-HubEnvFile "D:\dftb-neu\data\classroom\secrets.env"
if (Test-Path "D:\cmats-lab\.env") {
    Get-Content "D:\cmats-lab\.env" | ForEach-Object {
        if ($_ -match "^CMATS_DEEPSEEK_API_KEY=(.*)$" ) { $env:DFTB_CLASS_DEEPSEEK_API_KEY =
$Matches[1].Trim() }
        if ($_ -match "^CMATS_LLM_API_KEY=(.*)$" ) { $env:DFTB_CLASS_LLM_API_KEY =
$Matches[1].Trim() }
        if ($_ -match "^CMATS_MP_API_KEY=(.*)$" ) { $env:DFTB_CLASS_MP_API_KEY =
$Matches[1].Trim() }
    }
}

```

```

    if ($_ -match "^CMATS_LLM_PROVIDER=(.*)$") { $env:DFTB_CLASS_LLM_PROVIDER =
$Matches[1].Trim() }
    if ($_ -match "^CMATS_DEEPSEEK_BASE_URL=(.*)$") { $env:DFTB_CLASS_DEEPSEEK_BASE_URL
= $Matches[1].Trim() }
    if ($_ -match "^CMATS_DEEPSEEK_MODEL=(.*)$") { $env:DFTB_CLASS_DEEPSEEK_MODEL =
$Matches[1].Trim() }
    if ($_ -match "^CMATS_LLM_BASE_URL=(.*)$") { $env:DFTB_CLASS_LLM_BASE_URL =
$Matches[1].Trim() }
    if ($_ -match "^CMATS_LLM_MODEL=(.*)$") { $env:DFTB_CLASS_LLM_MODEL =
$Matches[1].Trim() }
  }
}
if (-not $env:DFTB_CLASS_LLM_PROVIDER) { $env:DFTB_CLASS_LLM_PROVIDER = "deepseek" }

New-Item -ItemType Directory -Force -Path "D:\dftb-neu\data\classroom" | Out-Null
New-Item -ItemType Directory -Force -Path "D:\dftb-neu\data" | Out-Null

$py = "D:\cmats-lab\.venv\Scripts\python.exe"
if (-not (Test-Path $py)) { $py = "D:\anaconda3\python.exe" }

$outLog = "D:\dftb-neu\data\classroom_hub.out.log"
$errorLog = "D:\dftb-neu\data\classroom_hub.err.log"
"" | Set-Content -Path $outLog -Encoding utf8
"" | Set-Content -Path $errorLog -Encoding utf8

$argsList = @(
    "-m", "uvicorn", "classroom_hub.main:app",
    "--app-dir", "D:\dftb-neu\apps",
    "--host", "0.0.0.0",
    "--port", "8791"
)

$p = Start-Process -FilePath $py -ArgumentList $argsList -WorkingDirectory "D:\dftb-neu"
,
    -WindowStyle Hidden -PassThru -RedirectStandardOutput $outLog -RedirectStandardError
$errorLog
"started pid=$( $p.Id )" | Out-File -FilePath "D:\dftb-neu\data\classroom_hub.pid.txt" -
Encoding ascii
Write-Output "started pid=$( $p.Id )"
# ===== FILE: scripts/start_hub_funnel.ps1 =====
# One-shot: ensure classroom hub Funnel is up and write the fixed public URL.
$errorActionPreference = "Stop"
$root = Split-Path -Parent $PSScriptRoot
$urlFile = Join-Path $root "data\classroom\public_hub_url.txt"
$funnelUrl = if ($env:DFTB_FUNNEL_HUB_URL) { $env:DFTB_FUNNEL_HUB_URL.TrimEnd("/") }
else { "https://desktop-ibhgp7g.tai
lcc9705.ts.net" }
$hubPort = if ($env:DFTB_CLASS_PORT) { [int]$env:DFTB_CLASS_PORT } else { 8791 }
$remoteHost = if ($env:DFTB_FUNNEL_SSH_HOST) { $env:DFTB_FUNNEL_SSH_HOST } else
{ "4060" }

New-Item -ItemType Directory -Force -Path (Split-Path $urlFile) | Out-Null

```

```

Set-Content -Path $UrlFile -Value $FunnelUrl -Encoding utf8

function Test-LocalHub {
    try {
        $r = Invoke-WebRequest "http://127.0.0.1:$HubPort/health" -UseBasicParsing -
TimeoutSec 3
        return ($r.StatusCode -eq 200)
    } catch { return $false }
}

if (Test-LocalHub) {
    & tailscale funnel --bg --yes "http://127.0.0.1:$HubPort"
} else {
    $cmd = "tailscale funnel --bg --yes http://127.0.0.1:$HubPort"
    ssh -o BatchMode=yes -o ConnectTimeout=15 $RemoteHost $cmd
}

Write-Output "PUBLIC_HUB_URL=$FunnelUrl"
Write-Output "url_file=$UrlFile"

$py = Join-Path $Root ".venv\Scripts\python.exe"
if (Test-Path $py) {
    & $py -c @"
import httpx, certifi, sys
u=sys.argv[1]
r=httpx.get(u+'/health', timeout=25, verify=certifi.where(), trust_env=False)
print('health', r.status_code, r.text[:120])
sys.exit(0 if r.status_code==200 else 1)
"@ $FunnelUrl
} else {
    $h = Invoke-RestMethod "$FunnelUrl/health" -TimeoutSec 20
    Write-Output ("health_ok=" + $h.ok)
}

# ===== FILE: scripts/start_hub_public_tunnel.ps1 =====
# Deprecated: localhost.run temporary tunnel retired.
# Use Tailscale Funnel (fixed HTTPS) instead.
$ErrorActionPreference = "Stop"
Write-Warning "start_hub_public_tunnel.ps1 已停用 (localhost.run)。改走
scripts\start_hub_funnel.ps1"
& (Join-Path $PSScriptRoot "start_hub_funnel.ps1")
# ===== FILE: scripts/sync-dftb-laptop.ps1 =====
$ErrorActionPreference = "Continue"
$src = "C:\Users\13919\Projects\dftb-neu"
$stu = "C:\Users\13919\AppData\Local\Programs\dftb-neu\resources"
$tea = "C:\Users\13919\AppData\Local\Programs\dftb-neu-teacher\resources"

function Sync-Dir($from, $to) {
    if (-not (Test-Path $from)) { Write-Host "missing $from"; return }
    if (Test-Path $to) { Remove-Item -Recurse -Force $to }
    $parent = Split-Path $to
    if (-not (Test-Path $parent)) { New-Item -ItemType Directory -Force -Path $parent |
Out-Null }

```

```

Copy-Item -Recurse -Force $from $to
Write-Host "synced $from -> $to"
}

Get-ChildItem $src -Recurse -Force -Filter ".*" -ErrorAction SilentlyContinue |
    Remove-Item -Force -ErrorAction SilentlyContinue

Sync-Dir "$src\apps\web" "$stu\web"
Sync-Dir "$src\apps\api" "$stu\api"
Sync-Dir "$src\engines" "$stu\engines"
Sync-Dir "$src\templates" "$stu\templates"
Sync-Dir "$src\scripts" "$stu\scripts"
Sync-Dir "$src\apps\teacher_web" "$tea\teacher_web"

$helper = @'
param()
$src = "C:\Users\13919\Projects\dftb-neu"
$stu = "C:\Users\13919\AppData\Local\Programs\dftb-neu\resources"
$tea = "C:\Users\13919\AppData\Local\Programs\dftb-neu-teacher\resources"
function Sync-Dir($from, $to) {
    if (Test-Path $to) { Remove-Item -Recurse -Force $to }
    Copy-Item -Recurse -Force $from $to
    Write-Host "OK $from"
}
Sync-Dir "$src\apps\web" "$stu\web"
Sync-Dir "$src\apps\api" "$stu\api"
Sync-Dir "$src\engines" "$stu\engines"
Sync-Dir "$src\templates" "$stu\templates"
Sync-Dir "$src\scripts" "$stu\scripts"
Sync-Dir "$src\apps\teacher_web" "$tea\teacher_web"
Write-Host "DONE - restart DFTB apps"
'@
Set-Content -Path "$src\sync-to-installed.ps1" -Value $helper -Encoding UTF8

# Update desktop shortcuts to 0.2.1 installs
$WshShell = New-Object -ComObject WScript.Shell
$desk = [Environment]::GetFolderPath("Desktop")
$stuExe = "C:\Users\13919\AppData\Local\Programs\dftb-neu\DFTB 工作台.exe"
$teaExe = "C:\Users\13919\AppData\Local\Programs\dftb-neu-teacher\DFTB 教师端.exe"
# resolve actual exe names
$stuExe = (Get-ChildItem "C:\Users\13919\AppData\Local\Programs\dftb-neu\*.exe" |
Where-Object { $_.Name -notlike "Unins
tall*" } | Select-Object -First 1).FullName
$teaExe = (Get-ChildItem "C:\Users\13919\AppData\Local\Programs\dftb-neu-teacher\*.exe"
| Where-Object { $_.Name -notlik
e "Uninstall*" } | Select-Object -First 1).FullName
Write-Host "stuExe=$stuExe"
Write-Host "teaExe=$teaExe"
if ($stuExe) {
    $s = $WshShell.CreateShortcut("$desk\DFTB 工作台.lnk")
    $s.TargetPath = $stuExe
    $s.WorkingDirectory = Split-Path $stuExe

```

```

    $s.Save()
}
if ($teaExe) {
    $s = $WshShell.CreateShortcut("$desk\DFTB 教师端.lnk")
    $s.TargetPath = $teaExe
    $s.WorkingDirectory = Split-Path $teaExe
    $s.Save()
}

Write-Host "=== verify ==="
Write-Host ("web=" + (Test-Path "$stu\web\index.html"))
Write-Host ("api=" + (Test-Path "$stu\api\dftbneu\main.py"))
Write-Host ("teacher=" + (Test-Path "$tea\teacher_web\index.html"))
if (Test-Path "$tea\teacher_web\config.json") { Get-Content
"$tea\teacher_web\config.json" -Raw }
Write-Host DONE
# ===== FILE: scripts/sync_and_restart_hub_4060.ps1 =====
# 在 4060 (desktop-ibhgp7g) 上运行: 同步课堂中心代码并重启服务
# 用法 (管理员 PowerShell 可选):
# powershell -NoProfile -ExecutionPolicy Bypass -File D:\dftb-
neu\scripts\sync_and_restart_hub_4060.ps1
# 或先指定源码目录:
# $env:DFTB_NEU_SRC = 'C:\Users\13919\Projects\dftb-neu'

$erroractionpreference = "Stop"
$DestRoot = "D:\dftb-neu"
$SrcCandidates = @(
    $env:DFTB_NEU_SRC,
    "C:\Users\13919\Projects\dftb-neu",
    "D:\dftb-neu-src",
    (Join-Path $PSScriptRoot "..")
) | Where-Object { $_ -and (Test-Path $_) }

$Src = $null
foreach ($c in $SrcCandidates) {
    $hub = Join-Path $c "apps\classroom_hub\main.py"
    if (Test-Path $hub) { $Src = (Resolve-Path $c).Path; break }
}
if (-not $Src) {
    throw "未找到源码 (需含 apps\classroom_hub\main.py)。请设置 DFTB_NEU_SRC。"
}

Write-Host "SRC=$Src"
Write-Host "DST=$DestRoot"

New-Item -ItemType Directory -Force -Path "$DestRoot\apps\classroom_hub" | Out-Null
New-Item -ItemType Directory -Force -Path "$DestRoot\scripts" | Out-Null
New-Item -ItemType Directory -Force -Path "$DestRoot\data\classroom" | Out-Null

Copy-Item "$Src\apps\classroom_hub\*" "$DestRoot\apps\classroom_hub\" -Recurse -Force
Copy-Item "$Src\scripts\run_classroom_hub_4060.*" "$DestRoot\scripts\" -Force -
ErrorAction SilentlyContinue

```

```

Copy-Item "$Src\scripts\start_classroom_hub_4060.ps1" "$DestRoot\scripts\" -Force -
ErrorAction SilentlyContinue
Copy-Item "$Src\scripts\restart_classroom_hub_4060.bat" "$DestRoot\scripts\" -Force -
ErrorAction SilentlyContinue
Copy-Item "$Src\scripts\sync_and_restart_hub_4060.ps1" "$DestRoot\scripts\" -Force

if (-not (Select-String -Path "$DestRoot\apps\classroom_hub\main.py" -Pattern
"teacher/gate/clear" -Quiet)) {
    throw "同步后仍缺少 /teacher/gate/clear, 中止重启。"
}

Write-Host "Stopping listeners on 8791..."
Get-NetTCPConnection -LocalPort 8791 -State Listen -ErrorAction SilentlyContinue |
    ForEach-Object { Stop-Process -Id $_.OwningProcess -Force -ErrorAction
SilentlyContinue }
Start-Sleep -Seconds 2

$env:DFTB_CLASS_HOST = "0.0.0.0"
$env:DFTB_CLASS_PORT = "8791"
$env:DFTB_CLASS_DATA = "$DestRoot\data\classroom"
if (-not $env:DFTB_CLASS_TEACHER_PASSWORD) { $env:DFTB_CLASS_TEACHER_PASSWORD =
"z1303@" }
if (-not $env:DFTB_CLASS_ADMIN_SECRET) { $env:DFTB_CLASS_ADMIN_SECRET =
$env:DFTB_CLASS_TEACHER_PASSWORD }
if (-not $env:DFTB_CLASS_TOKEN_SECRET) { $env:DFTB_CLASS_TOKEN_SECRET =
$env:DFTB_CLASS_TEACHER_PASSWORD }

# 从本机 cmats-lab/.env 注入 DeepSeek 等密钥 (不打印)
foreach ($envFile in @("D:\cmats-lab\.env", "C:\Users\13919\Projects\cmats-lab\.env"))
{
    if (-not (Test-Path $envFile)) { continue }
    Get-Content $envFile | ForEach-Object {
        if ($_ -match "^CMATS_DEEPSEEK_API_KEY=(.+)") { $env:DFTB_CLASS_DEEPSEEK_API_KEY =
$Matches[1].Trim() }
        if ($_ -match "^CMATS_LLM_API_KEY=(.+)") { $env:DFTB_CLASS_LLM_API_KEY =
$Matches[1].Trim() }
        if ($_ -match "^CMATS_MP_API_KEY=(.+)") { $env:DFTB_CLASS_MP_API_KEY =
$Matches[1].Trim() }
        if ($_ -match "^CMATS_LLM_PROVIDER=(.+)") { $env:DFTB_CLASS_LLM_PROVIDER =
$Matches[1].Trim() }
        if ($_ -match "^CMATS_DEEPSEEK_BASE_URL=(.+)") { $env:DFTB_CLASS_DEEPSEEK_BASE_URL
= $Matches[1].Trim() }
        if ($_ -match "^CMATS_DEEPSEEK_MODEL=(.+)") { $env:DFTB_CLASS_DEEPSEEK_MODEL =
$Matches[1].Trim() }
        if ($_ -match "^CMATS_LLM_BASE_URL=(.+)") { $env:DFTB_CLASS_LLM_BASE_URL =
$Matches[1].Trim() }
        if ($_ -match "^CMATS_LLM_MODEL=(.+)") { $env:DFTB_CLASS_LLM_MODEL =
$Matches[1].Trim() }
    }
    break
}
if (-not $env:DFTB_CLASS_LLM_PROVIDER) { $env:DFTB_CLASS_LLM_PROVIDER = "deepseek" }

```

```

$pyCandidates = @(
    "D:\cmats-lab\.venv\Scripts\python.exe",
    "D:\anaconda3\python.exe",
    "C:\Users\13919\Projects\dftb-neu\.venv\Scripts\python.exe",
    (Get-Command python -ErrorAction SilentlyContinue | Select-Object -ExpandProperty
Source)
) | Where-Object { $_ -and (Test-Path $_) }
$py = $pyCandidates | Select-Object -First 1
if (-not $py) { throw "未找到 Python, 无法启动课堂中心。" }

$outLog = "$DestRoot\data\classroom_hub.out.log"
$errorLog = "$DestRoot\data\classroom_hub.err.log"
"" | Set-Content $outLog -Encoding utf8
"" | Set-Content $errorLog -Encoding utf8

$sargList = @(
    "-m", "uvicorn", "classroom_hub.main:app",
    "--app-dir", "$DestRoot\apps",
    "--host", "0.0.0.0",
    "--port", "8791"
)
$p = Start-Process -FilePath $py -ArgumentList $sargList -WorkingDirectory $DestRoot `
    -WindowStyle Hidden -PassThru -RedirectStandardOutput $outLog -RedirectStandardError
$errorLog
"started pid=$( $p.Id )" | Set-Content "$DestRoot\data\classroom_hub.pid.txt" -Encoding
ascii
Write-Host "started pid=$( $p.Id) python=$py"

Start-Sleep -Seconds 3
try {
    $h = Invoke-RestMethod "http://127.0.0.1:8791/health" -TimeoutSec 5
    Write-Host ("health ok llm_configured=" + $h.llm_configured + " provider=" +
$h.llm_provider)
} catch {
    Write-Host "health check failed: $($_.Exception.Message)"
    Write-Host "--- err log ---"
    Get-Content $errorLog -ErrorAction SilentlyContinue | Select-Object -Last 40
    throw
}

# 计划任务保活
schtasks /Delete /TN dftb-classroom-hub /F 2>$null | Out-Null
$str = "powershell.exe -NoProfile -ExecutionPolicy Bypass -File
`"$DestRoot\scripts\start_classroom_hub_4060.ps1`"
schtasks /Create /TN dftb-classroom-hub /TR $str /SC ONLOGON /RL LIMITED /F | Out-Null
Write-Host "sync_and_restart_done"
# ===== FILE: scripts/wsl_deploy_dftb.sh =====
#!/usr/bin/env bash
# DFTB Neu: 在 Linux / WSL 用户目录隔离部署 DFTB+ (不用 sudo)
set -euo pipefail
CMATS_ROOT="${DFTB_NEU_HOME:-$HOME/.dftb-neu}"

```

```

BIN_DIR="$CMATS_ROOT/bin"
ENV_DIR="$CMATS_ROOT/envs/dftbplus"
SK_ROOT="$CMATS_ROOT/share/dftb/sk"
JOB_ROOT="$CMATS_ROOT/jobs"
mkdir -p "$BIN_DIR" "$CMATS_ROOT/envs" "$CMATS_ROOT/tmp" "$SK_ROOT" "$JOB_ROOT"

echo "DEPLOY_ROOT=$CMATS_ROOT"

MM="$BIN_DIR/micromamba"
if [ ! -x "$MM" ]; then
    ARCH=$(uname -m)
    case "$ARCH" in
        x86_64|amd64) MM_ARCH=linux-64 ;;
        aarch64|arm64) MM_ARCH=linux-aarch64 ;;
        *) echo "UNSUPPORTED_ARCH=$ARCH"; exit 2 ;;
    esac
    URL="https://micro.mamba.pm/api/micromamba/${MM_ARCH}/latest"
    TMP="$CMATS_ROOT/tmp/micromamba.tar.bz2"
    if command -v curl >/dev/null 2>&1; then
        curl -fsSL "$URL" -o "$TMP"
    else
        wget -qO "$TMP" "$URL"
    fi
    mkdir -p "$CMATS_ROOT/tmp/mmextract"
    # 精简 Ubuntu/WSL 往往没有 bzip2; 优先用系统 tar, 失败则用 python3 解压
    if tar -xjf "$TMP" -C "$CMATS_ROOT/tmp/mmextract" bin/micromamba 2>/dev/null; then
        :
    elif command -v python3 >/dev/null 2>&1; then
        python3 - "$TMP" "$CMATS_ROOT/tmp/mmextract" <<'PY'
import sys, tarfile
src, dest = sys.argv[1], sys.argv[2]
with tarfile.open(src, "r:bz2") as tf:
    members = [m for m in tf.getmembers() if m.name.endswith("bin/micromamba") or
m.name == "bin/micromamba"]
    if not members:
        members = [m for m in tf.getmembers() if m.name.endswith("micromamba")]
    tf.extractall(dest, members=members or None)
PY
    else
        echo "NEED_BZIP2_OR_PYTHON3"
        exit 3
    fi
    if [ -f "$CMATS_ROOT/tmp/mmextract/bin/micromamba" ]; then
        mv "$CMATS_ROOT/tmp/mmextract/bin/micromamba" "$MM"
    else
        found=$(find "$CMATS_ROOT/tmp/mmextract" -type f -name micromamba | head -n 1 ||
true)
        if [ -n "$found" ]; then
            mv "$found" "$MM"
        else
            echo "MICROMAMBA_EXTRACT_FAIL"
            exit 3

```

```

    fi
fi
chmod +x "$MM"
echo "MICROMAMBA_INSTALLED"
fi

GLIBC_MAJ=$(ldd --version 2>&1 | head -n1 | grep -oE '[0-9]+\.[0-9]+' | head -n1 ||
echo 2.17)
GLIBC_OK_FLAG=1
case "$GLIBC_MAJ" in
    2.1[7-9]|2.2[0-7]|2.[0-9]|1.*) GLIBC_OK_FLAG=0 ;;
esac
echo "GLIBC=$GLIBC_MAJ modern=$GLIBC_OK_FLAG"

if [ -x "$ENV_DIR/bin/dftb+" ] && "$ENV_DIR/bin/dftb+" --version >/dev/null 2>&1; then
    echo "ENV_OK"
else
    if [ -e "$ENV_DIR" ]; then
        mv "$ENV_DIR" "${ENV_DIR}_broken_$$" 2>/dev/null || rm -rf "$ENV_DIR"
    fi
    if [ "$GLIBC_OK_FLAG" = "1" ]; then
        "$MM" create -y -p "$ENV_DIR" -c conda-forge 'dftbplus*=nomp*_*' dftbplus-tools || \
        \
        "$MM" create -y -p "$ENV_DIR" -c conda-forge dftbplus dftbplus-tools
    else
        "$MM" create -y -p "$ENV_DIR" -c conda-forge 'dftbplus=21.2' || \
        "$MM" create -y -p "$ENV_DIR" -c conda-forge 'dftbplus=22.2'
    fi
    echo "ENV_CREATED"
fi

download_sk() {
    local set="$1"
    local dest="$SK_ROOT/$set"
    if find "$dest" -name '*.skf' 2>/dev/null | grep -q .; then
        echo "SK_EXISTS=$set"
        return 0
    fi
    mkdir -p "$dest"
    local url="https://github.com/dftbparams/${set}/archive/refs/heads/main.tar.gz"
    local tar="$CMATS_ROOT/tmp/${set}.tar.gz"
    curl -fsSL "$url" -o "$tar" || wget -qO "$tar" "$url" || return 1
    mkdir -p "$CMATS_ROOT/tmp/sk_$set"
    tar -xzf "$tar" -C "$CMATS_ROOT/tmp/sk_$set"
    find "$CMATS_ROOT/tmp/sk_$set" -name '*.skf' -exec cp -f {} "$dest/" \;
    n=$(find "$dest" -name '*.skf' | wc -l | tr -d ' ')
    echo "SK_OK=$set count=$n"
}

for s in 3ob mio matsci pbc; do
    download_sk "$s" || echo "SK_FAIL=$s"
done

```

```

export PATH="$ENV_DIR/bin:$PATH"
echo "FEATURE_PROBE_BEGIN"
dftb+ --version 2>&1 | head -n 8 || true
which dftb+
echo "SK_TOTAL=$(find "$SK_ROOT" -name '*.skf' | wc -l | tr -d ' ')"
echo "FEATURE_PROBE_END"

# H2O 测试校验 (Recipes)
SMOKE="$JOB_ROOT/smoke_h2o"
rm -rf "$SMOKE"
mkdir -p "$SMOKE"
cat > "$SMOKE/geo.gen" <<'GEN'
3 C
O H
  1   1   0.000000000000E+00 -1.000000000000E+00  0.000000000000E+00
  2   2   0.000000000000E+00  0.000000000000E+00  0.783064000000E+00
  3   2   0.000000000000E+00  0.000000000000E+00 -0.783064000000E+00
GEN
SK3OB="$SK_ROOT/3ob"
cat > "$SMOKE/dftb_in.hsd" <<HSD
Geometry = GenFormat {
  <<< "geo.gen"
}
Hamiltonian = DFTB {
  SCC = Yes
  ThirdOrderFull = Yes
  HubbardDerivs {
    O = -0.1575
    H = -0.1857
  }
  MaxAngularMomentum {
    O = "p"
    H = "s"
  }
  SlaterKosterFiles = Type2FileNames {
    Prefix = "${SK3OB}/"
    Separator = "-"
    Suffix = ".skf"
  }
  Filling = Fermi {
    Temperature [K] = 300
  }
}
Driver = GeometryOptimization {
  Optimizer = Rational {}
  MaxSteps = 50
  OutputPrefix = "geo_end"
  Convergence = {
    GradElem = 1E-4
  }
}

```

```
Options {
    WriteDetailedOut = Yes
}
InputVersion = "24.1"
HSD
(cd "$SMOKE" && dftb+ > dftb.log 2>&1) || true
if grep -q "Geometry converged" "$SMOKE/detailed.out" 2>/dev/null; then
    echo "SMOKE_OK"
else
    echo "SMOKE_FAIL"
    tail -40 "$SMOKE/dftb.log" || true
fi
echo "DEPLOY_OK"
```